Formalization of a Nuclear Power Plant Life-Cycle Model for AI and Machine Learning Applications

Yuriy Dmitrishin

yu.dmitrishin@gmail.com

2025

Abstract

This paper presents a formal, research-oriented model designed to support a broad range of artificial intelligence and machine learning (AI/ML) tasks related to managing the life cycle of nuclear power plants (NPPs). The model consists of a hierarchical graph structure, a semantic layer, and a classification system. It also includes links to a corpus and associated metrics. It is intended primarily to advance research on AI/ML methods that support NPP life cycle management, including engineering, construction, commissioning and startup, and decommissioning. At the same time, with appropriate adaptation and optimization of selected components and algorithms, the model can be adapted for deployment in real NPP projects. The model does not have to be implemented in full and can be adapted, simplified or extended to meet the specific objectives of AI/ML applications for implementation in NPP projects. Certain findings and recommendations presented herein may necessitate additional investigation, depending on the specific research objectives or on how the model is implemented for particular applications. While this work concentrates on the life cycle of nuclear power plants, the proposed principles and methods are transferable to other complex systems.

Introduction

This section defines the scope and organizing principles of the model. We introduce five interoperable components—M, S, D, Q, and U—that together provide a consistent basis for representing structure and classifications, aligning semantics, binding external evidence, quantifying attributes and metrics, and coordinating operations across layers. The focus is on clear roles, minimal coupling between layers, and invariants that support validation and reuse across projects. The components are summarized below.

- M — Structure. A multi-level representation of the project that also includes the hierarchical class systems for objects and connections; it separates structural decomposition from dependencies and keeps classifications consistent across levels.
- S — Semantics. Ontological and linguistic domain model. It organizes domain concepts and relation types, term categories and lexical mappings, and associated

attributes and constraints; it supports reasoning, consistency checks, and terminology management, and aligns with M via bidirectional links rather than redefining its classifications.

- D — Corpus. Typed references from model elements to source documents and datasets, enabling traceability and evidence-based validation.
- Q — Quantitative layer. Typed attributes and metrics, with simple aggregation rules, to compute readiness, risk, cost, and other values for analysis and as features for AI/ML.
- U — Meta-level. An orchestration layer that coordinates M, S, D, Q and, in its ML extension, governs dataset generation, training, and evaluation while preserving model consistency.

The following sections describe these components and the consistency rules that bind them together. They also explain how selected parts can be adapted for research prototypes and, through targeted engineering, for practical use in NPP projects.

1. Basic Structure of the Graph M_0

The basic structure of the model is defined by the tuple

$$M_0 = (L, V, E_{\text{hor}}, E_{\text{vert}}, \theta),$$

where

- $L = \{1, 2, \ldots, k\}$ is an ordered set of hierarchy levels;
- V is a set of vertices;
- $\theta : V \to L$ is a mapping that assigns each vertex $v \in V$ to exactly one level $l \in L$;
- $E_{\text{hor}} \subseteq \big\{(v, W) \;\big|\; v \in V,\ W \subseteq V \setminus \{v\},\ W \neq \varnothing\big\}$ is the set of *horizontal hyperedges*; here, v is the *source* of the hyperedge, and W is the *target*, a non-empty subset of vertices at the *same level* as v;
- $E_{\text{vert}} \subseteq V \times V$ is the set of *vertical edges* connecting vertices of adjacent levels.

Assumption. The sets of levels and vertices are non-empty:

$$L \neq \varnothing, \qquad V \neq \varnothing.$$

Notation.

- $\mathcal{P}^+(X)$ denotes the set of all non-empty subsets of a set X;
- $(v, W) \in E_{\text{hor}}$ is read as "a hyperedge from vertex v to the set of vertices W";
- w will denote an arbitrary element of the set W.

1.1 Invariants

1. **Vertex Level Uniqueness:**

$$\forall v \in V \ \exists! l \in L : \ \theta(v) = l.$$

2. **Horizontal Hyperedges:**

$$\forall (v, W) \in E_{\text{hor}} \ \big[v \notin W \ \wedge \ \forall w \in W : \theta(w) = \theta(v) \big].$$

3. **Vertical Edges:**

$$\forall (v, w) \in E_{\text{vert}} : \ \theta(w) = \theta(v) + 1.$$

4. **Edge Level (extension of θ):**

$$\theta\big((v, W)\big) := \theta(v), \qquad \theta\big((v, w)\big) := \theta(v).$$

1.2 Terminology

- *Horizontal hyperedge* — a pair $(v, W) \in E_{\text{hor}}$, where $v \in V$, $W \subseteq V \setminus \{v\}$, $W \neq \varnothing$, and $\theta(w) = \theta(v)$ for all $w \in W$;
- *Vertical edge* — a pair $(v, w) \in E_{\text{vert}}$, where $\theta(w) = \theta(v) + 1$;
- *Aggregation* — the mapping

$$\mu_V : V \to \mathcal{P}(V), \qquad \mu_V(v) := \{ w \in V \mid (v, w) \in E_{\text{vert}} \}.$$

2. Level Structure, Aggregation, and Connectivity

2.1 Level Organization

The set of levels is denoted by $L = \{1, 2, \ldots, k\}$, where 1 is the top level of the hierarchy, and k is the bottom level. The levels are ordered increasingly: $1 < 2 < \cdots < k$.

Each structural element $y \in Y := V \cup E_{\text{hor}} \cup E_{\text{vert}}$ is assigned to exactly one level:

$$\forall y \in Y \ \ \exists!\, l \in L : \theta(y) = l.$$

The initial mapping $\theta : V \to L$ is extended to the edges by the following rule:

- for a horizontal hyperedge $e = (v, W) \in E_{\text{hor}}$

$$\theta(e) := \theta(v);$$

- for a vertical edge $e = (v, w) \in E_{\text{vert}}$

$$\theta(e) := \theta(v).$$

This extension is consistent with the model's invariants:

$$\forall (v, W) \in E_{\text{hor}}, \ \forall w \in W : \ \theta(w) = \theta(v), \qquad \forall (v, w) \in E_{\text{vert}} : \ \theta(w) = \theta(v) + 1.$$

Invariant: No outgoing vertical edges at the last level. If $\theta(v) = k = \max L$, then there exists no $w \in V$ such that $(v, w) \in E_{\text{vert}}$.
Remark (flat case) If $L = \{1\}$, then $E_{\text{vert}} = \varnothing$ and the model reduces to a single horizontal hypergraph $G(1) = (V, E_{\text{hor}})$ subject to Inv-HorAcyc (and, if required, weak connectivity); the aggregation μ_V and the decomposition δ are unused.

2.2 Horizontal Hyperedges

A horizontal hyperedge (hereafter, a *hyperedge*) is an element $(v, W) \in E_{\text{hor}}$, where

- $v \in V$ is the source vertex,
- $W \subseteq V \setminus \{v\}, \quad W \neq \varnothing$ is a non-empty set of target vertices, *not containing* v,
- $\theta(w) = \theta(v)$ for all $w \in W$.

The condition $v \notin W$ prevents self-loops and is consistent with Invariant 2 of Section 1.

Note on the hypergraph interpretation. The pair (v, W) can be interpreted as a directed hyperarc from a single source vertex to a set of target vertices. In this model, such edges are localized within a single level; therefore, the full potential of general hypergraph theory is not utilized.

2.3 Edge Types

The model distinguishes between two logically and functionally independent types of edges:

- **Horizontal hyperedges** — elements of the set

$$E_{\text{hor}} \subseteq \big\{ (v, W) \;\big|\; v \in V,\ W \subseteq V \setminus \{v\},\ W \neq \varnothing \big\},$$

  that define connections *within a single level*. Each such hyperedge $(v, W) \in E_{\text{hor}}$ connects a source vertex v to a non-empty set of target vertices W, such that

$$\forall w \in W : \quad \theta(w) = \theta(v).$$

  The horizontal hyperedges at level l form a subgraph

$$\begin{aligned} G^{(l)} &:= \big(V^{(l)}, E^{(l)}\big), \\ V^{(l)} &:= \{\, v \in V \mid \theta(v) = l \,\}, \\ E^{(l)} &:= \big\{\, (v, W) \in E_{\text{hor}} \mid \theta(v) = l \,\big\}. \end{aligned}$$

  (The condition $\theta(w) = l$ for $w \in W$ is ensured by a global invariant.)

- **Vertical edges** — elements of the set $E_{\text{vert}} \subseteq V \times V$ that define hierarchical connections between levels. Each edge $(v, w) \in E_{\text{vert}}$ connects a vertex v at level l to a vertex w at level $l + 1$:

$$\theta(w) = \theta(v) + 1.$$

These edges define the aggregation mapping

$$\mu_V : V \longrightarrow \mathcal{P}(V), \qquad \mu_V(v) \ := \ \{\, w \in V \mid (v, w) \in E_{\text{vert}} \}.$$

The following equivalence holds: $(v, w) \in E_{\text{vert}} \iff w \in \mu_V(v)$.

Note. The mapping μ_V forms the *aggregation structure* of the model: each vertex v at level l is associated with a subset of vertices at level $l + 1$ that are aggregated into v.

Vertical edges do not participate in the construction of the horizontal subgraphs $G^{(l)}$ and are not included in the definition of intra-level reachability.

Horizontal hyperedges and vertical edges are disjoint and describe different aspects of the structure: the former describe connectivity within a level, while the latter describe the hierarchy between levels. This separation prevents conflicts in the interpretation of connections and ensures a strict hierarchical organization of the model.

2.4 Aggregation and Decomposition

Each vertex $v \in V$ at level l can be detailed by a *decomposition subgraph* at the next level, $l + 1$. This decomposition is defined by the mapping

$$\delta : V \longrightarrow \big\{\, (V_{\text{sub}}, E_{\text{sub}}) \,\big\}, \qquad v \ \longmapsto \ \delta(v) \ = \ (V_{\text{sub}}, E_{\text{sub}}),$$

where

- $V_{\text{sub}} \subseteq V^{(l+1)} := \{\, w \in V \mid \theta(w) = l + 1 \,\}$ is the set of vertices at level $l + 1$ that constitute the decomposition subgraph; in this case,

  $$\mu_V(v) \ \subseteq \ V_{\text{sub}},$$

  meaning that all vertices *directly aggregated* into v (by definition, $\mu_V(v) = \{\, w \in V \mid (v, w) \in E_{\text{vert}} \}$) are included in the decomposition. The presence of additional (*auxiliary*) vertices $V_{\text{sub}} \setminus \mu_V(v)$ is permitted.

- $E_{\text{sub}} \subseteq \big\{\, (x, Y) \in E_{\text{hor}} \mid x \in V_{\text{sub}},\ Y \subseteq V_{\text{sub}} \setminus \{x\} \big\}$ is the set of horizontal hyperedges, each of which lies entirely within V_{sub}.

Closure of the decomposition. For any hyperedge $(x, Y) \in E_{\text{sub}}$, the conditions $x \in V_{\text{sub}}$ and $Y \subseteq V_{\text{sub}} \setminus \{x\}$ hold. This property ensures the logical locality of the subgraph and simplifies the verification of the model's correctness.

Role of vertical edges. The vertical edges $(v, w) \in E_{\text{vert}}$, originating from v to the vertices $w \in \mu_V(v) \subseteq V_{\text{sub}}$, formalize the aggregation connections between levels and serve as the "external interface" of the decomposition.

Auxiliary vertices. Let $\delta(v) = (V_{\text{sub}}, E_{\text{sub}})$. The vertices in the set $V_{\text{sub}} \setminus \mu_V(v)$ are called *auxiliary* within this decomposition.

- Each auxiliary vertex w retains all the global properties of level $l+1$ and can act as an aggregating vertex for level $l+2$ via edges $(w, z) \in E_{\text{vert}}$.
- The same vertex w can be used in the decompositions of different vertices $v_1, v_2, \ldots$ — this is useful for modeling shared components.
- Incoming edges $(u, w) \in E_{\text{vert}}$ from other vertices $u \in V^{(l)}$ $(u \neq v)$ are possible: an auxiliary vertex can be aggregated by multiple parents.
- Conversely, by the definition of $\mu_V(v)$, the existence of an edge $(v, w) \in E_{\text{vert}}$ for $w \in V_{\text{sub}} \setminus \mu_V(v)$ is *prohibited* to avoid confusing the auxiliary and directly aggregated roles.

Thus, the decomposition $\delta(v)$ reveals the internal structure of vertex v through a locally closed subgraph at the next level, where only a subset of vertices, $\mu_V(v)$, is connected to v by vertical edges, while the remaining vertices serve as auxiliary components that can be freely used in other parts of the model.

2.5 Paths and Reachability on a Horizontal Level

Consider a fixed level $l \in L$ and its horizontal hypergraph:

$$G^{(l)} = \big(V^{(l)}, E^{(l)}\big), \qquad E^{(l)} = \{\, (v, W) \in E_{\text{hor}} \mid \theta(v) = l \,\}.$$

An element (v, W) will be referred to as a *horizontal hyperedge*.

Definition 2.1 (Directed Hyperpath)**.** A sequence

$$P = \big(v_0,\, (v_0, W_0),\, v_1,\, (v_1, W_1),\, \ldots,\, (v_{m-1}, W_{m-1}),\, v_m\big), \qquad m \geq 0,$$

is called a *directed hyperpath* in $G^{(l)}$ if, for each $i \in \{0, \ldots, m-1\}$, the following conditions hold:

$$(v_i, W_i) \in E^{(l)} \quad \text{and} \quad v_{i+1} \in W_i.$$

The vertex v_0 is called the *source*, v_m is the *sink* of the path, and m is the *length* of P.

Definition 2.2 (Reachability)**.** For a vertex $v \in V^{(l)}$, we define the set of reachable vertices as:

$$\text{Reach}_l^+(v) := \big\{\, u \in V^{(l)} \,\big|\, \exists \text{ a hyperpath } P : v \rightsquigarrow u \text{ of length } m \geq 1 \big\}.$$

Remark. The reflexive-transitive closure is defined as $\text{Reach}_l^*(v) := \{v\} \cup \text{Reach}_l^+(v)$. If $|W| = 1$ for all $(\cdot, W) \in E^{(l)}$, then Definitions 2.1–2.2 reduce to the classical notions of a path and reachability in a directed graph.

Invariant (Inv-HorAcyc)

$$(v, W) \in E_{\text{hor}} \implies \text{Reach}_{\theta(v)}^+(v) \cap W = \varnothing.$$

Level Acyclicity. The level hypergraph $G^{(l)}$ is called *acyclic* if

$$\forall v \in V^{(l)} : \quad v \notin \mathrm{Reach}_l^+(v) \text{ (see Definition 2.2).}$$

Weak Connectivity. Consider the underlying undirected graph $\mathrm{UG}^{(l)} = (V^{(l)}, E_u)$, where $\{x, y\} \in E_u$ if and only if there exists a hyperedge $(x, W) \in E^{(l)}$ with $y \in W$ or $(y, W) \in E^{(l)}$ with $x \in W$. We say that $G^{(l)}$ is *weakly connected* if the undirected graph $\mathrm{UG}^{(l)}$ is connected.

3. Structural Level Acyclicity and Connectivity

For each level $l \in L$, the horizontal hypergraph

$$G^{(l)} = \left(V^{(l)}, E^{(l)}\right)$$

is considered with respect to two key structural properties:

- **Acyclicity.** In $G^{(l)}$, there must be no directed hyperpath of non-zero length that starts and ends at the same vertex.
- **Weak Connectivity.** If the direction of hyperedges in $G^{(l)}$ is disregarded, the resulting underlying undirected graph must be connected: any pair of vertices must belong to a single connected component.

Note on Application Flexibility. The requirement for the graph $G^{(l)}$ to be weakly connected is recommended as a baseline condition for the structural integrity of the level, especially when $G^{(l)}$ is interpreted as a functionally cohesive block. However, in certain application scenarios, it may be appropriate to relax this requirement—for instance, by allowing multiple disconnected components within a level, provided that this does not compromise the consistency of other model mechanisms (e.g., aggregation, morphisms, iteration). In such cases, the model's correctness must be ensured by additional constraints, and any deviation from connectivity must be explicitly documented in the implementation specification.

Note on Validation. Maintaining the structural requirements for $G^{(l)}$ (acyclicity and, where applicable, weak connectivity) is the responsibility of the procedures that modify the set of horizontal edges $E_{\mathrm{hor}}^{(l)}$ at level l. Verification of these properties can be performed either upon each addition or removal of an edge or as part of a final validation of the level's configuration.

4. Classification of Vertices and Edges

The model introduces the following classification mappings:

$$\kappa_V : V \to \mathcal{P}(C_V), \qquad \kappa_E : E_{\mathrm{hor}} \cup E_{\mathrm{vert}} \to \mathcal{P}(C_E)$$

where:

- C_V is the set of vertex classes or categories;

- C_E is the set of edge classes;
- $\kappa_V(v) \subseteq C_V$ is the set of classification features for vertex v;
- $\kappa_E(e) \subseteq C_E$ is the set of features for an edge $e \in E_{\text{hor}} \cup E_{\text{vert}}$.

Note on the structure of classes. In practical implementations of the model, each element c of a class set (e.g., C_V or C_E) can represent not merely an atomic label, but a structured record. Each class c may be described by an extended set of annotations:

$$c \;=\; \big(key,\, \{\langle value_j,\, tag_j\rangle\}_{j=1}^{m}\big),$$

where

- $\boldsymbol{key} \in Val_{\text{str}}$ is a unique string identifier that serves as the formal representative of class c within the model; it is this identifier that all invariants and mappings (e.g., κ_V) refer to;
- $value_j \in Val$ is an arbitrary atomic annotation value (e.g., a string, number, date, boolean, or range) associated with class c;
- $tag_j \in Val_{\text{str}}$ is a text tag describing the semantics or purpose of the corresponding $value_j$ (e.g., `description`, `short_name`, `color`).

The number of annotations m and their content are not specified by the base model. All formal invariants (e.g., hierarchical consistency checks) rely exclusively on the ***key*** component.

4.1 Multi-level Classifications

The classifications are organized as hierarchical trees:

$$\mathcal{H}_V = (C_V, \preceq_V), \quad \mathcal{H}_E = (C_E, \preceq_E)$$

where $\preceq_V$ and $\preceq_E$ are partial order relations that define the class hierarchy.

Each class $c \in C_V \cup C_E$ can be:

- a root class (at the highest level),
- an intermediate class (an internal node in the hierarchy),
- or a leaf class (having no subclasses).

Classification by both leaf and intermediate classes is permitted. The mappings κ_V and κ_E can contain both general and specialized features, corresponding to different levels of the hierarchy.

Note on edge classification. Edge classification covers both horizontal hyperedges and vertical (hierarchical) connections, which are defined in the model by the sets E_{hor} and E_{vert}, respectively. Horizontal edges connect vertices on the same hierarchy level, whereas vertical edges establish connections between vertices on adjacent levels.

5. Semantic Morphism

5.1 Class Hierarchies

To link the structural and semantic components of the model, each graph element is associated with *classification features*, organized into two (in general, independent) class hierarchies:

$$\mathcal{H}_V = \big(C_V,\ \preceq_V\big), \qquad \mathcal{H}_E = \big(C_E,\ \preceq_E\big),$$

where

- C_V is a finite set of vertex classes (categories);
- C_E is a finite set of edge classes (categories);
- $\preceq_V \subseteq C_V \times C_V$ and $\preceq_E \subseteq C_E \times C_E$ are partial order relations that form rooted trees (each non-root class has a unique parent).

Definition 5.1 (Special Class for Vertical Edges)**.** Within the set of edge classes C_E, a special class c_{vert} is used to denote vertical (hierarchical) connections between levels. All vertical edges in the model must belong to this class c_{vert} or its subclasses.

Note on the tree-like structure of class hierarchies. The partial order relations $\preceq_V$ and $\preceq_E$, defined for the hierarchies $\mathcal{H}_V = (C_V, \preceq_V)$ and $\mathcal{H}_E = (C_E, \preceq_E)$, satisfy two additional properties:

(T1) *Single Root:* In each class set, there exists exactly one element with no strict predecessors.

(T2) *Unique Parent:* Each non-root class has exactly one immediate predecessor.

Invariants (T1)–(T2) ensure that each hierarchy forms a rooted tree, which is a special case of a directed acyclic graph (DAG). All previously introduced definitions and invariants remain unchanged.

Classification Mappings. Each vertex and each edge is assigned a set of classes (which may be empty) from the corresponding hierarchy:

$$\kappa_V : V \longrightarrow \mathcal{P}(C_V), \qquad \kappa_E : E_{\text{hor}} \cup E_{\text{vert}} \longrightarrow \mathcal{P}(C_E),$$

where $\mathcal{P}(\cdot)$ denotes the powerset. Thus,

- $\kappa_V(v) \subseteq C_V$ is the set of classes to which vertex $v \in V$ belongs;
- $\kappa_E(e) \subseteq C_E$ is the set of classes to which edge $e \in E_{\text{hor}} \cup E_{\text{vert}}$ belongs.

Note on forests and a virtual root. In practical implementations, a class hierarchy often represents a *forest*—a collection of several independent trees $T_1, \ldots, T_k$ with roots $r_1, \ldots, r_k$. To preserve the "single root" invariant **(T1)** without altering the existing relations between classes, the forest can be formally reduced to a single tree by adding a single *virtual root* $\top_H$:

$$\top_H \preceq_V r_i, \quad i = 1, \ldots, k.$$

The extended order $\preceq_V^*$, which includes the relations $\top_H \preceq_V^* r_i$, does not affect the rest of the forest's structure and preserves all formulated invariants. This representation allows for the uniform application of algorithms designed for a tree-like structure, even when the original classification consists of several independent trees.

Remarks.

1. The presence of multiple classes in $\kappa_V(v)$ or $\kappa_E(e)$ reflects the polymorphic nature of the element or accommodates alternative semantic interpretations.

2. An empty set is permissible and is interpreted as the absence of an explicit classification at the current stage of the model's construction.

3. The relations $\preceq_V$ and $\preceq_E$ can be arbitrary partial orders; however, in most practical scenarios, they form rooted trees, which simplifies the verification of classification consistency (see this section).

5.2 Morphism Definition

A semantic morphism φ represents a mapping from graph elements (vertices and edges) to the corresponding classes in the hierarchies. Formally, it is defined as a pair of mappings:

$$\varphi = (\varphi_V, \varphi_E), \quad \text{where} \quad \varphi_V \colon V \to C_V, \quad \varphi_E \colon E_{\text{hor}} \cup E_{\text{vert}} \to C_E.$$

For each vertex $v \in V$, the value $\varphi_V(v)$ represents a specific class from the set C_V, selected among the admissible classes in $\kappa_V(v)$. Similarly, for each edge $e \in E_{\text{hor}} \cup E_{\text{vert}}$, the value $\varphi_E(e)$ is selected from $\kappa_E(e)$.

Remark. The morphism φ facilitates the transition from a multi-label classification (where an element may correspond to multiple admissible classes) to a single semantic interpretation within a specific context of the model's application.

5.3 Consistency Conditions

A morphism φ is considered *admissible* if it satisfies the following conditions:

1. $\varphi_V(v) \in \kappa_V(v)$ for all $v \in V$.

2. $\varphi_E(e) \in \kappa_E(e)$ for all $e \in E_{\text{hor}} \cup E_{\text{vert}}$.

3. For any pair $(v, w) \in E_{\text{vert}}$: $\varphi_V(w) \preceq_V \varphi_V(v)$, i.e., the class of the child vertex w is a subclass of (or the same as) the class of the aggregating vertex v.

4. For any vertical edge $(v, w) \in E_{\text{vert}}$: $\varphi_E((v, w)) \preceq_E c_{\text{vert}}$, where $c_{\text{vert}} \in C_E$ is the special class denoting vertical connections.

Remark (Note on Consistency Conditions). Conditions (3) and (4) are crucial for maintaining the semantic integrity of the model's hierarchical structure. Condition (3) ensures consistency between aggregating vertices and their children with respect to the vertex class hierarchy $\mathcal{H}_V$. Condition (4) enforces the correct typing of vertical connections within the edge class hierarchy $\mathcal{H}_E$ by classifying them under the special class c_{vert} or its subclasses.

Remark (Explanation of Condition 3 (Consistency of Vertex Aggregation)). This condition ensures that the semantics of the child vertex w (class $\varphi_V(w)$) is consistent with the semantics of the parent vertex v (class $\varphi_V(v)$) according to the hierarchy $\preceq_V$. If v aggregates w, then the class of w must be more specialized than or the same as the class of v.

Remark (Explanation of Condition 4 (Typing of Vertical Edges)). The class c_{vert} is a special element of the edge class set C_E. It labels all vertical (hierarchical) connections in the model. The requirement $\varphi_E((v, w)) \preceq_E c_{\text{vert}}$ means that any vertical edge is classified as c_{vert} or one of its subclasses.

5.4 Purpose of the Morphism

The semantic morphism φ plays a crucial role in establishing the link between the structural and semantic components of the model. It is used for the following primary purposes:

- **Consistency Checking:** Verifying the consistency between the structural elements of the graph (vertices, edges, and their connections) and their assigned semantic classes (elements from C_V and C_E), taking into account the hierarchies $\mathcal{H}_V$ and $\mathcal{H}_E$ and the consistency conditions (see §5.3).
- **Enforcing Hierarchical Constraints:** Ensuring compliance with the semantic constraints imposed by the model's hierarchical structure, such as the consistency of classes upon aggregation (Condition 3 in §5.3) and the typing of vertical connections (Condition 4 in §5.3).
- **Automated Model Validation:** Using the formal consistency conditions to automatically verify the correctness of the model's semantic annotation. Errors or inconsistencies in the morphism can indicate an incorrect application of the classification scheme or data errors.
- **Foundation for Analysis and Data Processing:** Generating semantically annotated graph representations that serve as input for subsequent tasks, including:
  - automatic classification of elements based on their semantic features and connections;
  - semantically-oriented aggregation and generalization of information;
  - training artificial intelligence and machine learning models on graph data;
  - performing optimization tasks that consider the semantics of the system components.

5.5 The Set of Admissible Morphisms

Instead of considering a single semantic mapping $\varphi : M_0 \to (\mathcal{H}_V, \mathcal{H}_E)$, the model focuses on *the set of all admissible morphisms*, denoted by Φ. This set comprises all mappings satisfying the consistency conditions with respect to the classification $\kappa = (\kappa_V, \kappa_E)$ and the hierarchies $\mathcal{H}_V = (C_V, \preceq_V)$ and $\mathcal{H}_E = (C_E, \preceq_E)$.

Definition. The set of admissible morphisms Φ is defined as the collection of all morphisms $\varphi = (\varphi_V, \varphi_E)$ satisfying the consistency conditions specified in §5.3:

$$\Phi = \left\{ \varphi = (\varphi_V, \varphi_E) \;\middle|\; \begin{array}{l} \varphi_V : V \to C_V, \quad \varphi_E : E_{\text{hor}} \cup E_{\text{vert}} \to C_E; \\ \forall v \in V : \; \varphi_V(v) \in \kappa_V(v); \\ \forall e \in E_{\text{hor}} \cup E_{\text{vert}} : \; \varphi_E(e) \in \kappa_E(e); \\ \forall (v,w) \in E_{\text{vert}} : \; \varphi_V(w) \preceq_V \varphi_V(v); \\ \forall (v,w) \in E_{\text{vert}} : \; \varphi_E((v,w)) \preceq_E c_{\text{vert}} \end{array} \right\},$$

where $c_{\text{vert}} \in C_E$ is the special class denoting vertical edges.

Semantics of the set Φ.

- **Space of Interpretations:** If, for some graph elements, the set $\kappa_V(v)$ or $\kappa_E(e)$ contains multiple admissible classes (i.e., $|\kappa_V(v)| > 1$ or $|\kappa_E(e)| > 1$), then the set Φ describes the entire space of possible semantic interpretations (morphisms) that are consistent with the model. Each $\varphi \in \Phi$ represents one such specific interpretation scenario.

- **Indicator of Semantic Configuration Correctness:** If $\Phi = \varnothing$, it signifies that there exists no morphism satisfying all the consistency conditions. Consequently, the model's semantic configuration (including the sets κ_V, κ_E and the hierarchies $\mathcal{H}_V, \mathcal{H}_E$) is incorrect or contradictory.

  Remark (On the case $\Phi = \varnothing$). The case $\Phi = \emptyset$ indicates that no semantic morphism φ satisfies the specified consistency conditions. This typically points to an inconsistency or incompleteness in the model's semantic configuration (e.g., incompatible classes in $\kappa_V(v)$ relative to the $\preceq_V$ hierarchy, empty sets of admissible classes, or logical conflicts in κ). It is important to note that the basic structural graph model M_0 may still be syntactically correct. In such cases, it is recommended to diagnose the semantic layer of the model specifically.

  Invariant (Inv-Φ)

  $$\Phi \neq \varnothing.$$

- **A Tool for Analysis and Selection:** The set Φ can be used for:
  - analyzing alternative semantic scenarios;
  - selecting the most relevant morphism φ^* based on additional criteria (see the paragraph on the evaluation function and optimal morphism selection below);
  - validating the semantic completeness and logical consistency of the model (consistency is verified by checking that $\Phi \neq \varnothing$).

Evaluation Function and Optimal Morphism Selection. To select an optimal morphism from the set Φ if it contains more than one element, an evaluation function $f_{\text{eval}} : \Phi \to \mathbb{R}_{\geq 0}$ is used. This function assigns a numerical value to each morphism $\varphi \in \Phi$, reflecting its quality or relevance for a specific task. This allows for the formalization of the choice of an optimal morphism φ^*:

$$\varphi^* = \arg\min_{\varphi \in \Phi} f_{\text{eval}}(\varphi) \quad \big(\text{or } \arg\max_{\varphi \in \Phi} f_{\text{eval}}(\varphi), \text{ depending on the semantics of } f_{\text{eval}}\big).$$

The function f_{eval} is not a fixed part of the general model; its specification is determined during the application of the model to a specific task. Possible criteria for constructing the function f_{eval} include:

- consistency with a priori knowledge or preferred semantic patterns;
- completeness of coverage of model elements with significant classes (e.g., minimizing undefined or overly general classes);
- the degree of specialization (depth) of the assigned classes in the hierarchies $\mathcal{H}_V, \mathcal{H}_E$;
- consistency with external data or knowledge sources (e.g., with the text corpus D or ontologies);
- probabilistic or statistical metrics (e.g., if classes are associated with probabilities).

Remark (Interpretation of the Evaluation Function f_{eval} and Choice of Optimization). The interpretation of the evaluation function $f_{\text{eval}} : \Phi \to \mathbb{R}_{\geq 0}$ and, consequently, the choice between minimization and maximization depends on the semantics attributed to $f_{\text{eval}}(\varphi)$, where we assume the function is monotonic with respect to the partial order $\preceq$ defined on Φ.

- If $f_{\text{eval}}(\varphi)$ reflects the *cost*, *complexity*, *error*, or degree of *generality* of a morphism, then **minimization** is used to select the optimal φ^*:
$$\varphi^* = \arg\min_{\varphi \in \Phi} f_{\text{eval}}(\varphi).$$
- If $f_{\text{eval}}(\varphi)$ represents *utility*, *quality*, *specificity*, *confidence*, or degree of *conformance* to target requirements, then **maximization** is used:
$$\varphi^* = \arg\max_{\varphi \in \Phi} f_{\text{eval}}(\varphi).$$

The specific choice of the criterion and the optimization direction must be explicitly defined and justified in any practical application of the model.

Note on selecting a specific class in cases of multiplicity in $\kappa(x)$. The model assumes that the classification functions κ_V and κ_E can assign multiple admissible classes to graph elements. Constructing a specific morphism $\varphi \in \Phi$ requires selecting *one* class from the corresponding set $\kappa(x)$ for each element $x \in V \cup E_{\text{hor}} \cup E_{\text{vert}}$, such that all consistency conditions are met.

In a practical implementation, especially with automated generation of morphisms, it may be necessary to explicitly define a *selection strategy* for cases where $|\kappa(x)| > 1$. Possible strategies include:

- using a priori priorities assigned to the classes;
- selecting the class that minimizes or maximizes its depth in the hierarchy ($\mathcal{H}_V$ or $\mathcal{H}_E$);
- applying heuristics based on external context or the properties of the element x itself.

It is important to understand that such selection strategies do not affect the formal definition of the set Φ itself (which includes all theoretically admissible morphisms), but they may be necessary for the practical application of the model—for instance, to select one "best" or "default" morphism from Φ.

Example of an evaluation function. As an evaluation function f_{eval}, one could use, for example, the average depth of the classes assigned to vertices:

$$f_{\text{eval}}(\varphi) = \frac{1}{|V|} \sum_{v \in V} \text{depth}_V\big(\varphi_V(v)\big),$$

where $\text{depth}_V(c)$ denotes the depth of class $c \in C_V$ in the hierarchical tree $\mathcal{H}_V$ (e.g., the root has a depth of 0 or 1). Minimizing such a function would favor the selection of morphisms with more general (less specialized) vertex classes.

Remark (single-level case Let $L = \{1\}$. Then $\theta : V \to \{1\}$ is constant and, by the vertical-edge rule $\theta(w) = \theta(v) + 1$, we have $E_{\text{vert}} = \varnothing$; thus the aggregation μ_V and the decomposition δ are unused. The class hierarchies H_V, H_E, the multi-label mappings κ_V, κ_E, and the admissible morphism set Φ remain applicable; the consistency conditions of §5.3 that quantify over E_{vert} are vacuous, while all other invariants and typed-attribute mechanisms operate unchanged.

6. Vertex and Edge Attributes

The model allows for associating additional characteristics—*attributes*—with the vertices and edges of a graph. Attributes represent properties, parameters, measurements, or other contextual data necessary for detailed analysis, semantic classification, performing computations, or training models.

6.1 Attribute Types and Values

Each attribute is formally defined by an ordered pair (τ, val), where:

- $\tau \in \text{AType}$ is the *application-specific type* of the attribute, which defines its nature and permissible operations. The set AType will be defined later in Section §6.3.
- val is the *specific value* of the attribute, which must be consistent with its application-specific type τ. Formally, $\text{val} \in \text{Dom}_\tau$, where Dom_τ is the domain of values for type τ.

The attribute typing system is built upon a hierarchy of base types, universes of values for these types, and application-specific types, along with their respective value domains.

Base Value Types. We define a set of *base (or primitive) types*, denoted as VType, which serves as the foundation for attribute typing. This set may include, but is not limited to, the following types:

$$\text{VType} = \big\{\mathbb{R},\ \mathbb{Z},\ \text{Bool},\ \text{Cat}(C),\ \text{Enum}(E),\ \text{Distr}(P),\ \text{Struct}(S),\ \mathbb{R}^n, \dots\big\},$$

where:

- $\mathbb{R}$ — real numbers;
- $\mathbb{Z}$ — integers;
- Bool — Boolean type;

- $\mathrm{Cat}(C)$ — categorical type, defined by a set C;
- $\mathrm{Enum}(E)$ — enum type, defined by a set E;
- $\mathrm{Distr}(P)$ — probability distribution, defined by a family P;
- $\mathrm{Struct}(S)$ — structured type, based on a schema S;
- $\mathbb{R}^n$ — a vector of real numbers of dimension n.

The set VType is extensible.

Universes of Values for Base Types. For each base type $v \in \mathrm{VType}$, a non-empty set Val_v is strictly fixed, called the *universe of values* for type v. This set comprises all theoretically permissible values for the given base type.

Examples:

- $\mathrm{Val}_{\mathbb{R}} = \mathbb{R}$ — all real numbers.
- $\mathrm{Val}_{\mathrm{Bool}} = \{\mathrm{true}, \mathrm{false}\}$ — Boolean values.
- $\mathrm{Val}_{\mathrm{Cat}(C)} = C$ — the set of categories C.
- $\mathrm{Val}_{\mathrm{Enum}(E)} = E$ — the set of elements in enumeration E.

When a new base type v' is added to VType, its universe of values $\mathrm{Val}_{v'}$ must also be defined simultaneously.

Relation to Application-Specific Attribute Types. For more specific typing within the context of an application domain, a set of *application-specific attribute types*, AType, is introduced. Each application-specific type $\tau \in \mathrm{AType}$ is based on some base type. Formally, there exists a mapping

$$\beta\colon \mathrm{AType} \to \mathrm{VType},$$

that associates each application-specific type with its base type $\beta(\tau)$.

For each application-specific type τ, its *domain of values* Dom_τ is defined, which is a subset of the universe of values of the corresponding base type:

$$\mathrm{Dom}_\tau \subseteq \mathrm{Val}_{\beta(\tau)}.$$

The domain Dom_τ can impose additional constraints on the values compared to $\mathrm{Val}_{\beta(\tau)}$ (for example, a range for a numeric type). This three-tiered separation, "base type $\to$ universe of values $\to$ application-specific type domain," is used for type correctness verification and performing aggregate computations.

The set VType and the corresponding universes Val_v can be extended as needed, for instance, depending on the specifics of the application domain or the analysis tasks.

Remark (Correctness of Extending the Type System). When adding new types (base or application-specific) to the model's type system, a strict correctness check must be provided. Specifically, for each new application-specific type $\tau \in \mathrm{AType}$, the inclusion

$$\mathrm{Dom}_\tau \subseteq \mathrm{Val}_{\beta(\tau)}$$

must always hold. A violation of this condition will lead to type errors. The specific rules and mechanisms for extending the sets of types VType, AType, and their domains are determined by the policy of a specific implementation of the model and are beyond the scope of the general formalization presented here.

6.2 Attribute Mappings

Each element of the graph (a vertex or an edge) can be associated with a *set of typed attributes*. This set is a collection of "application-specific type–value" pairs. To establish this association, two mappings are introduced:

$$\mathrm{attr}_V \colon V \to \mathcal{P}(A), \qquad \mathrm{attr}_E \colon E_{\mathrm{hor}} \cup E_{\mathrm{vert}} \to \mathcal{P}(A),$$

where:

- A is the *universal set of all possible attributes*, that is,

$$A = \{\, (\tau, \mathrm{val}) \mid \tau \in \mathrm{AType} \text{ and } \mathrm{val} \in \mathrm{Dom}_\tau \,\}.$$

- $\mathcal{P}(A)$ is the power set of A, i.e., the set of all subsets of A.
- $\mathrm{attr}_V(v) \subseteq A$ is the set of attributes assigned to vertex $v \in V$.
- $\mathrm{attr}_E(e) \subseteq A$ is the set of attributes assigned to edge $e \in E_{\mathrm{hor}} \cup E_{\mathrm{vert}}$.

Thus, the functions attr_V and attr_E map each graph element to its own (possibly empty) subset of the attribute set A.

Type-Correctness of Attributes. Each attribute belonging to the sets $\mathrm{attr}_V(v)$ or $\mathrm{attr}_E(e)$ is a pair (α, val), where:

- $\alpha \in \mathrm{AType}$ is the application-specific type of the attribute;
- $\mathrm{val} \in \mathrm{Dom}_\alpha$ is the attribute value, belonging to the domain of type α.

This definition guarantees that the type correctness invariant holds: for any attribute (α, val) assigned to a graph element, the following condition is satisfied:

$$\mathrm{val} \in \mathrm{Dom}_\alpha \subseteq \mathrm{Val}_{\beta(\alpha)}.$$

6.3 Attribute Typing

The attribute type system is formalized using the following definitions:

- AType — the set of *application-specific attribute types*;
- VType — the set of *base value types* (see §6.1).

The central mapping associates application-specific types with base types:

$$\beta \colon \mathrm{AType} \to \mathrm{VType},$$

which maps each application-specific type $\tau \in \mathrm{AType}$ to a base type $\beta(\tau) \in \mathrm{VType}$.

Domain of an Application Type. For each application-specific type $\tau \in \mathrm{AType}$, its *domain of permissible values*, Dom_τ, is defined. This domain is a subset of the universe of values, $\mathrm{Val}_{\beta(\tau)}$, of the base type $\beta(\tau)$:

$$\mathrm{Dom}_\tau \subseteq \mathrm{Val}_{\beta(\tau)}.$$

(The universe of values Val_v for a base type v is defined in §6.1.)

The Set of Attributes. The set A of all *type-correct attributes* is defined as the union of all pairs of an application-specific type and a value from its domain:

$$A := \bigcup_{\tau \in \mathrm{AType}} \{ (\tau, \mathrm{val}) \mid \mathrm{val} \in \mathrm{Dom}_\tau \}.$$

Type-Correctness Invariant. From the definition of A, the fundamental *type-correctness invariant* directly follows: if $(\tau, \mathrm{val}) \in A$, then

$$\mathrm{val} \in \mathrm{Dom}_\tau.$$

Since $\mathrm{Dom}_\tau \subseteq \mathrm{Val}_{\beta(\tau)}$, it also holds that $\mathrm{val} \in \mathrm{Val}_{\beta(\tau)}$.

6.4 Usage of Attribute Typing

The introduced attribute typing system ensures:

- **Data Verification:** The correctness of each attribute (τ, val) (in $\mathrm{attr}_V(v)$ or $\mathrm{attr}_E(e)$) is verified by the condition
$$\mathrm{val} \in \mathrm{Dom}_\tau.$$
This prevents type errors and data incompatibilities.
- **Semantic Rigor:** By constraining values to predefined domains Dom_τ, typing precludes the assignment of logically incorrect or meaningless characteristics to graph elements.
- **Model Extensibility:** The previously defined sets of types $(\mathrm{VType}, \mathrm{AType})$ and their domains (Dom_τ) can be flexibly extended and adapted to the specifics of the application domain without compromising the model's integrity.
- **Foundation for Analysis and AI:** Typed attributes serve as a reliable input for data analysis and machine learning tasks, including filtering, aggregation, semantic matching, and feature engineering for AI models.

7. Attribute Aggregates and Semantic Generalizations

7.1 Purpose and Position in the Model

Attribute aggregates and semantic generalization mechanisms are an extension of the base model, designed for the automated derivation of new knowledge and characteristics from existing data and the graph structure. Their primary objectives include:

- **Data Aggregation (Computation of Derived Attributes):** Defining or refining attribute values for *aggregating vertices* (parent nodes in the hierarchy). These values are computed based on the attribute sets of their *subordinate (child) elements*—vertices and/or edges—that are associated with the aggregating vertex through the model's hierarchical structure (specifically, via the aggregation mapping μ_V and the vertical edges E_{vert}).

- **Semantic Generalization (Derivation of New Knowledge):** Forming new, derived classifications (denoted, for example, as κ') for graph elements, or generating new, semantically meaningful attributes. These generalizations are constructed based on the results obtained during the data aggregation process and may reflect higher-level properties or patterns.
- **Integration with Analytical Tasks and Decision Support:** The active use of aggregated attributes and derived semantic classifications in more complex model components or external analytical procedures. This includes their application in evaluation functions for semantic morphisms (e.g., the function f_{eval} from §5.5), in model validation and verification procedures, and in other tasks of intelligent data analysis and decision support.

A key requirement for the aggregation and semantic generalization mechanism is its *consistent integration* into the overall model architecture. This means that all derived attributes and classifications must strictly adhere to the typing system (§6) and must not violate the previously established structural and semantic invariants of the model.

7.2 Aggregate Functions

Let Λ be the set of aggregate functions, each of which is associated with a base result type and a list of permissible input types.

Signature.

$$\alpha\colon \mathcal{P}(A) \longrightarrow \mathrm{Val}_{\mathrm{ResType}(\alpha)},$$

where A is the set of all permissible attribute pairs (τ, val).

Type Compatibility Condition. For each function $\alpha \in \Lambda$ and any subset $I \subseteq A$,

$$\alpha(I) \in \mathrm{Val}_{\mathrm{ResType}(\alpha)}.$$

Type-Correctness Invariant for Aggregate Functions.

$$\forall \alpha \in \Lambda,\ \forall I \subseteq A:\ \alpha(I) \in \mathrm{Val}_{\mathrm{ResType}(\alpha)}.$$

Aggregate Function Specification. For each function $\alpha \in \Lambda$, we define:

$$\mathrm{InBase}(\alpha) \in \mathrm{VType}, \quad \mathrm{ResType}(\alpha) \in \mathrm{VType},$$

and, optionally,

$$\mathrm{AcceptedTypes}(\alpha) \subseteq \mathrm{AType}.$$

If $\mathrm{AcceptedTypes}(\alpha)$ is not specified, all types τ such that $\beta(\tau) = \mathrm{InBase}(\alpha)$ are permissible.

Let $I \subseteq A$ be the set of attributes provided as input to α. We define the filtered set

$$\mathrm{Input}_\alpha = \left\{ (\tau, \mathrm{val}) \in I \;\middle|\; \begin{array}{l} \big(\mathrm{AcceptedTypes}(\alpha) = \varnothing \text{ or } \tau \in \mathrm{AcceptedTypes}(\alpha)\big) \\ \wedge\ \beta(\tau) = \mathrm{InBase}(\alpha) \end{array} \right\}.$$

Complete Type Compatibility Condition. If $\text{Input}_\alpha \neq \varnothing$, then

$$\alpha(\text{Input}_\alpha) \in \text{Val}_{\text{ResType}(\alpha)}.$$

If $\text{Input}_\alpha = \varnothing$, the default value (`default_val`) is returned (or the computation is considered invalid).

Remark. In the presence of special values (`NaN`, `Infinity`), the function α must handle them according to a chosen strategy (e.g., exclude, replace, or raise an error).

Specification Example.
SUM: $\text{InBase} = \mathbb{R},\ \text{ResType} = \mathbb{R},\ \text{AcceptedTypes} = \varnothing.$
AND: $\text{InBase} = \text{Bool},\ \text{ResType} = \text{Bool}.$

Function α	InBase(α)	ResType(α)	Example Use-Case
SUM	$\mathbb{R}$	$\mathbb{R}$	Sum of costs
AVG	$\mathbb{R}$	$\mathbb{R}$	Average time
AND	Bool	Bool	Readiness of all nodes
COUNT	Any†	$\mathbb{Z}$	Number of components

† *The 'Any' label indicates that 'COUNT' accepts input attributes of any base type $v \in \text{VType}$, as it only counts their number.*

Remark. The behavior of α_{count} for mixed types in Input_α is implementation-defined.

7.3 Aggregation Rules and Contexts

Attribute aggregation is governed by a set of *aggregation rules*, Σ_{agg}. Each rule $r \in \Sigma_{\text{agg}}$ is specified by the tuple

$$r = \left(\tau^{(r)}_{\text{target}},\ \text{ContextSelector}_r,\ \text{AttributeSelector}_r,\ \alpha_r,\ \text{default_val}_r\right),$$

where

- $\tau^{(r)}_{\text{target}} \in \text{AType}$ is the application-specific type of the aggregated attribute created by rule r;
- ContextSelector_r is a function that, for a target element elem, determines the set of *source elements* for aggregation (e.g., descendants $\mu_V(\text{elem})$ or associated edges);
- $\text{AttributeSelector}_r$ is a filter function that selects from the attributes of the elements, specified by $\text{ContextSelector}_r(\text{elem})$, those pairs $(\tau, \text{val}) \in A$ that are permissible for aggregation by function α_r;
- $\alpha_r \in \Lambda$ is the aggregate function applied to the selected set of attributes;
- $\text{default_val}_r \in \text{Val}_{\text{ResType}(\alpha_r)}$ is the default value used when the input set for α_r is empty.

Computation of the Aggregated Value. For a target element elem and a rule r, the following steps are performed:

1. **Data Selection.**

$$\mathrm{Input}_{\alpha_r}(\mathrm{elem}) = \mathrm{AttributeSelector}_r\Big(\mathcal{A}\big(\mathrm{ContextSelector}_r(\mathrm{elem})\big),\ \alpha_r\Big),$$

   where $\mathcal{A}(\cdot)$ denotes the union of attributes from the selected set of elements.

2. **Aggregation.**

$$\mathrm{val}^{(r)}_{\mathrm{target}} = \begin{cases} \alpha_r\big(\mathrm{Input}_{\alpha_r}(\mathrm{elem})\big), & \mathrm{Input}_{\alpha_r}(\mathrm{elem}) \neq \varnothing, \\ \mathrm{default_val}_r, & \mathrm{Input}_{\alpha_r}(\mathrm{elem}) = \varnothing. \end{cases}$$

Default Handling Strategies. If $\mathrm{Input}_{\alpha_r}(\mathrm{elem}) = \varnothing$ and $\mathrm{default_val}_r$ is not specified, the result is considered undefined. The COUNT function, by default, returns the number of (τ, val) pairs—that is, $|\mathrm{Input}_{\alpha_r}(\mathrm{elem})|$—regardless of their base types $\beta(\tau)$. The parameters C, E for the types $\mathrm{Cat}(C)$ and $\mathrm{Enum}(E)$ are specified when declaring $\tau \in \mathrm{AType}$.

Rule Conflict Resolution. If multiple rules with the same $\tau^{(r)}_{\mathrm{target}}$ are applicable to a single element, the specific implementation must define a resolution mechanism (e.g., priority p_r, first in order, etc.).

Aggregation-Dependency DAG Constraint. The dependency graph of the rules in Σ_{agg} must form a directed acyclic graph (DAG).

7.4 Integration of Aggregated Attributes into the Model

The aggregated attributes, computed according to the rules from Σ_{agg}, are combined with the original attributes of the graph elements to form an *extended set of attributes*.

- Let $\mathrm{attr}(elem)$ be the original set of attributes for an element $elem \in V \cup E_{\mathrm{hor}} \cup E_{\mathrm{vert}}$ (i.e., $\mathrm{attr}_V(elem)$ or $\mathrm{attr}_E(elem)$).
- The extended set of attributes $\mathrm{attr}^*(elem)$ is defined as

$$\mathrm{attr}^*(elem) := \mathrm{attr}(elem) \ \cup \left\{ \left(\tau^{(r)}_{\mathrm{target}},\ \mathrm{val}^{(r)}_{\mathrm{target}}\right) \ \middle|\ \begin{array}{l} r \in \Sigma_{\mathrm{agg}} \text{ and rule } r \\ \text{is applicable to } elem \end{array} \right\},$$

  where

  - $\left(\tau^{(r)}_{\mathrm{target}}, \mathrm{val}^{(r)}_{\mathrm{target}}\right)$ is the type-value pair computed by rule r (see §7.3).

Key Properties of the Integration.

- **Preservation of Type Correctness.** For any pair $(\tau, \mathrm{val}) \in \mathrm{attr}^*(elem)$, the condition $\mathrm{val} \in \mathrm{Dom}_\tau$ holds.

- **Conflict Resolution.** A conflict arises if an aggregated attribute $\big(\tau^{(r)}_{\text{target}}, \text{val}^{(r)}_{\text{target}}\big)$ and an existing attribute $\big(\tau^{(r)}_{\text{target}}, \text{val}_{\text{local}}\big)$ from $\text{attr}(elem)$ share the same application-specific type $\tau^{(r)}_{\text{target}}$. The implementation must define a resolution strategy (e.g., prioritizing the existing value, overwriting, prohibition, etc.).

- **Monotonicity.** The original set of attributes is a subset of the extended set:

$$\text{attr}(elem) \subseteq \text{attr}^*(elem).$$

7.5 Extended Attribute Mapping for Vertices

The final (extended) set of attributes, $\text{attr}^*(v)$, for each vertex $v \in V$ is formed by the union of its locally defined attributes $\text{attr}(v)$ and the set of aggregated attributes computed according to the applicable rules $r \in \Sigma_{\text{agg}}$.

Let

$$\text{Types}\big(\text{attr}(v)\big) := \big\{\, \tau' \;\big|\; \exists\, \text{val}' : (\tau', \text{val}') \in \text{attr}(v) \big\}$$

denote the set of application-specific attribute types already present in $\text{attr}(v)$. Then,

$$\text{attr}^*(v) := \text{attr}(v) \cup \left\{ \big(\tau^{(r)}_{\text{target}}, \text{val}^{(r)}_{\text{target}}\big) \;\middle|\; \begin{array}{l} r \in \Sigma_{\text{agg}},\ r \text{ applicable to } v, \\ \tau^{(r)}_{\text{target}} \notin \text{Types}\big(\text{attr}(v)\big), \\ \text{val}^{(r)}_{\text{target}} \text{ has been successfully computed by } r \end{array} \right\}.$$

where

- $\big(\tau^{(r)}_{\text{target}}, \text{val}^{(r)}_{\text{target}}\big)$ is the type-value pair computed by rule r (see §7.3);
- a successful computation means that

$$\text{Input}_{\alpha_r}(v) \neq \varnothing \quad \text{or} \quad \text{default_val}_r \text{ was applied,}$$

  where

$$\text{Input}_{\alpha_r}(v) = \text{AttributeSelector}_r\big(\mathcal{A}(\text{ContextSelector}_r(v)),\ \alpha_r\big).$$

Invariants of the Extended Set $\text{attr}^*(v)$**.**

1. **Type Correctness.** For all $(\tau, \text{val}) \in \text{attr}^*(v)$, the condition $\text{val} \in \text{Dom}_\tau$ holds.
2. **Priority of Local Attributes.** Local attributes are not overwritten by aggregated ones, which is guaranteed by the condition $\tau^{(r)}_{\text{target}} \notin \text{Types}\big(\text{attr}(v)\big)$.
3. **Monotonicity.** $\text{attr}(v) \subseteq \text{attr}^*(v)$.

Remark (Alternative Conflict Resolution Strategies). The definition above implements a "local attributes take precedence" strategy. If necessary, other rules (e.g., merging, overwriting) can be applied; in which case, the construction step for $\text{attr}^*(v)$ must be modified accordingly.

Conflict-Resolver

Mode	Semantics
`PRIORITY`	Local value takes precedence over the aggregated one
`MERGE`	Merge by key, preserving both
`OVERRIDE`	Aggregated value replaces the local one

7.6 Semantic Generalizations Based on Aggregates

Let

$$\kappa' : V \longrightarrow \mathcal{P}(C')$$

be a derived classification of vertices into a new set of classes C', defined based on aggregated attributes. Then,

$$\kappa'(v) := \big\{\, c \in C' \mid \Pr\big(c, v, \mathrm{attr}^*(v)\big) = \mathrm{true} \big\},$$

where Pr is the predicate indicating whether the vertex belongs to the class, based on the extended attributes $\mathrm{attr}^*(v)$.

Hierarchical Inclusion.

$$\forall s \in \mu_V(v) : \quad \kappa'(s) \subseteq \kappa'(v).$$

Variants for Relaxing the Inheritance Condition for κ'.

1. **Hierarchical (subclass–superclass).**
   A partial order $\preceq_{C'}$ on C' is defined. For each $s \in \mu_V(v)$ and each $c_s \in \kappa'(s)$, there exists a $c_v \in \kappa'(v)$ such that $c_s \preceq_{C'} c_v$.

2. **Relaxed Intersection.**
   If $\kappa'(s) \neq \varnothing$, then $\kappa'(s) \cap \kappa'(v) \neq \varnothing$. The order $\preceq_{C'}$ is optional in this case.

3. **Parameterized Mode (adopted in this paper).**
   A global flag is introduced: `SemInheritanceMode`$\in \{$`STRICT`, `LOOSE`$\}$:

   (a) `STRICT`: inheritance is strict, $\kappa'(s) \subseteq \kappa'(v)$ (the default);

   (b) `LOOSE`: the rule from Variant 1 is applied (requires the $\preceq_{C'}$ hierarchy).

Option 3 is adopted, as it ensures backward compatibility (`STRICT`) and provides flexibility for a more fine-grained class hierarchy when needed (`LOOSE`).

7.7 Integration with Evaluation Functions

Aggregated attributes participate in the computation of the target metric

$$f_{\mathrm{eval}}(\varphi) = h\Big(\varphi, \bigcup_{v \in V} \mathrm{attr}^*(v), \bigcup_{e \in E_{\mathrm{hor}} \cup E_{\mathrm{vert}}} \mathrm{attr}^*(e)\Big),$$

where h is an application-specific function that implements the chosen quality criterion (e.g., consistency, completeness, preferences, etc.).

7.8 Invariants Related to Aggregation

Global Parameter. The inheritance mode for semantic classes is assumed to be fixed for the entire document:

$$\texttt{SemInheritanceMode} \in \{\texttt{STRICT},\ \texttt{LOOSE}\}, \qquad \text{defaulting to } \texttt{STRICT}.$$

1. **Acyclicity of Vertical Edges.**
   The hierarchical subgraph (V, E_{vert}) remains a directed acyclic graph (DAG): aggregation and decomposition do not create back edges or form cycles.

2. **Consistency of Semantic Generalizations.**
   For each vertex $v \in V$ and any of its immediate descendants $s \in \mu_V(v)$, a condition holds, depending on the chosen inheritance mode:

   `STRICT:` $\kappa'(s) \subseteq \kappa'(v)$.

   `LOOSE:` A partial order $\preceq_{C'}$ is defined on C', and

   $$\forall c_s \in \kappa'(s)\ \exists c_v \in \kappa'(v) :\ c_s \preceq_{C'} c_v.$$

   If the order $\preceq_{C'}$ is not defined, the mode automatically falls back to `STRICT`.

3. **Compatibility with Semantic Morphisms.**
   Let the class $\varphi_V(v)$ impose a constraint predicate P_φ on the set of permissible attributes. Then, for any attribute pair $(\tau, \text{val}) \in \text{attr}^*(v)$, the following holds:

   $$P_\varphi\big(\varphi_V(v),\ \tau,\ \text{val}\big) = \text{true}.$$

7.8.1 Type Correctness of Aggregate Functions

Let $\alpha \in \Lambda$ be an aggregate function with the specification

$$\begin{aligned} \text{InBase}(\alpha) &\in \text{VType}, \\ \text{ResType}(\alpha) &\in \text{VType}, \\ \text{AcceptedTypes}(\alpha) &\subseteq \text{AType (optional parameter)}. \end{aligned}$$

For an arbitrary subset of attributes $I \subseteq A$, we define the following (repeating the definition from § 7.2 for self-containment):

$$\text{Input}_\alpha(I) := \left\{ (\tau, \text{val}) \in I \;\middle|\; \begin{array}{l} \big(\text{AcceptedTypes}(\alpha) = \varnothing \ \vee\ \tau \in \text{AcceptedTypes}(\alpha)\big) \\ \wedge\ \beta(\tau) = \text{InBase}(\alpha) \end{array} \right\}.$$

(**C1**) If $\text{Input}_\alpha(I) \neq \varnothing$, then the result lies in the value universe of the result base type:

$$\alpha\big(\text{Input}_\alpha(I)\big) \ \in\ \text{Val}_{\text{ResType}(\alpha)}.$$

(**C2**) If $\text{Input}_\alpha(I) = \varnothing$, the value of $\alpha(\,\cdot\,)$ is undefined. When α is used in a rule $r \in \Sigma_{\text{agg}}$, an empty input is replaced by the value $\text{default_val}_r \in \text{Val}_{\text{ResType}(\alpha)}$ (see § 7.3); in its absence, the computation is considered invalid.

Compliance with conditions (**C1**)–(**C2**) ensures that the aggregate functions do not violate the model's type integrity.

7.9 Combined Classification

Let $\langle p_1, \ldots, p_m \rangle$ be an ordered sequence of previously defined features, where each p_i is either a typed attribute $\tau \in \mathrm{AType}$ or a class $c \in C_V$.

We impose a sequence of conditions $(p_1, \ldots, p_k)$ for $k = 1, \ldots, m$ on the vertices V. This forms a nested chain of sets:

$$V = V_0 \supseteq V_1 \supseteq \cdots \supseteq V_m \subseteq V,$$

which represents the levels of a combined hierarchy.

Note on value checking: – for an attribute, the value $\mathrm{attr}(v, \tau)$ is used; – for a class, the condition holds if $c \in \kappa_V(v)$.

For any metric f, the aggregates are computed in a standard manner:

$$\Sigma_{\mathrm{agg}}\big(V_k, f\big), \qquad k = 0, \ldots, m.$$

This approach supports arbitrary multi-level grouping based on existing attributes and/or classes without introducing new entities.

7.9.1 Generation of Combined Classes

For each specific combination of feature values $(p_1, \ldots, p_m)$, an additional class $c \in C_V$ can be introduced. This class is assigned to a vertex $v \in V$ if and only if v satisfies the specified set of conditions $(p_1, \ldots, p_m)$.

The resulting set of such classes forms a subtree in C_V, ordered by the same relation $\prec_V$ (prefix-based inclusion of combinations). This forms an independent hierarchical classification that can be used alongside the original ones without modifying other components of the model.

8. Iterative Processes within $G^{(l)}$ (Optional)

The Γ_{iter} component describes controlled iterative processes within the acyclic structure $G^{(l)}$ as an optional extension to the model. Iteration is realized by branching between a *statically unrolled* hyper-path and a direct bypass, thereby preserving acyclicity at level l.

$$\Gamma_{\mathrm{iter}} = \Big\{\gamma = \big(v_{\mathrm{entry}},\, v_{\mathrm{exit}},\, P_{\mathrm{loop}},\, P_{\mathrm{bypass}},\, N_{\max},\, \mathrm{Cond}\big)\Big\}.$$

- $v_{\mathrm{entry}}, v_{\mathrm{exit}} \in V^{(l)}$ are the entry and exit vertices of the iterative block;
- $P_{\mathrm{loop}} = (v_0, e_1, v_1, \ldots, e_m, v_m)$ is a *simple* directed hyper-path in $G^{(l)}$ (see Definition 2.1), with $v_0 = v_{\mathrm{entry}}$ and either $v_m = v_{\mathrm{exit}}$ or $(v_m, \{v_{\mathrm{exit}}\}) \in E_{\mathrm{hor}}$. By design, P_{loop} is the *static unrolling* of a loop of at most $N_{\max}$ steps (no cycles at level l);
- $P_{\mathrm{bypass}} = (v_{\mathrm{entry}}, \{\, v_{\mathrm{exit}} \,\}) \in E_{\mathrm{hor}}$ is a single horizontal hyper-edge providing a direct transition from v_{entry} to v_{exit} (the term "bypass path" is descriptive only);
- $N_{\max} \in \mathbb{N}$ is an upper bound on the number of steps in P_{loop};
- $\mathrm{Cond} : \mathrm{attr}^*(v_{\mathrm{entry}}) \to \{0, 1\}$ selects which structure is traversed at run time.

Remark. If the attributes required to evaluate Cond are absent from $\mathrm{attr}^*(v_{\mathrm{entry}})$, the implementation must explicitly define the behavior (e.g., return a default value, raise an error, or issue a warning). In this paper, the default is $\mathrm{Cond} = 0$.

Invariants

1. **Acyclicity.** P_{loop} is a simple hyper-path; P_{bypass} is a single hyper-edge; neither creates a cycle in $G^{(l)}$.
2. **Encapsulation.** All intermediate vertices of P_{loop} are encapsulated: entry is possible only through v_{entry}, exit only through v_{exit}.
3. **Bounded unfolding.** Let $|P_{\mathrm{loop}}| = m$ be the number of edges in P_{loop}. Then $m \leq N_{\max}$; transitions between steps are edges of $G^{(l)}$ (see Definition 2.1).
4. **Exclusive activation.** Exactly one structure is traversed at run time: P_{loop} if $\mathrm{Cond}(\mathrm{attr}^*(v_{\mathrm{entry}})) = 1$, otherwise P_{bypass}.
5. **Single level.** All vertices of $\gamma \in \Gamma_{\mathrm{iter}}$ belong to the same level:

$$\forall v, w \in \mathrm{Vertices}(\gamma) : \ \theta(v) = \theta(w) = l.$$

Purpose. The Γ_{iter} component extends the expressive power of the base model M_0 by allowing the description of controlled, repetitive processes while preserving the DAG properties. In its absence, the base model remains valid.

Notes on Iterative Processes The path $P_{\mathrm{loop}} = (v_0, \ldots, v_n, v_{\mathrm{exit}})$ represents a statically fixed sequence of vertices at level l: $v_0 = v_{\mathrm{entry}}$, and the final vertex v_n is either identical to v_{exit} or connected to it by a horizontal hyper-edge. The attributes of each $v_i \in P_{\mathrm{loop}}$ are given by $\mathrm{attr}(v_i)$; their extended variants $\mathrm{attr}^*(v_i)$ are computed using the rules from Σ_{agg}. By default,

$$\mathrm{Cond} : \mathrm{attr}^*(v_{\mathrm{entry}}) \to \{0, 1\}.$$

Recommendations for Modeling Dynamics The formal specification of Γ_{iter} does not prescribe a step-by-step processing of attributes within the loop. To model the dynamic evolution of states, the following approaches are possible:

- **Context-Dependent Aggregation.** For a vertex v_i, one can define a rule $r \in \Sigma_{\mathrm{agg}}$ with a $\mathrm{ContextSelector}_r(v_i)$ that includes preceding vertices (e.g., v_{i-1}) and their attributes.
- **Special Counter Attributes.** Introduce application-specific attributes that reflect the state of the iteration (counters, flags) and manage them through aggregation rules.

Clarification of Notation (Invariant 5). All vertices belonging to the component $\gamma \in \Gamma_{\mathrm{iter}}$ (i.e., v_{entry}, v_{exit}, and the vertices of P_{loop} and P_{bypass}) belong to the same level:

$$\forall v', w' \in \mathrm{Vertices}(\gamma) \ \Rightarrow \ \theta(v') = \theta(w'),$$

where $\mathrm{Vertices}(\gamma)$ is the set of all vertices appearing in the definition of γ.

9. Admissible Transformations

Base State. Let M_0 be a fixed initial (reference) state of the model; all possible states are considered within the same *fixed domain of graph element identifiers*. Clearly, $M_0 \in \mathcal{M}_{M_0}$; thus, the initial state itself satisfies all invariants and serves as a reference point for subsequent transformations.

State Signature. A model in an admissible state is defined by the tuple

$$M = (L, V, E_{\text{hor}}, E_{\text{vert}}, \theta, \kappa_V, \kappa_E, \text{attr}_V, \text{attr}_E, \Sigma_{\text{agg}}, \ldots).$$

All listed sets are assumed to be finite unless specified otherwise; this guarantees the decidability of the predicate $\text{Inv}(M)$. This tuple represents the complete specification of a single model configuration and defines a point in the multidimensional space of possible states. *Note.* The ellipsis "…" indicates the presence of other formally defined components (e.g., H_V, H_E, Φ, f_{eval}, Γ_{iter}), which can also be modified and are part of the model's state, as well as any subsequent extensions formally added to M and included in the predicate $\text{Inv}(M)$.

Correctness Invariant. Let

$$\text{Inv}(M)$$

denote the predicate that requires all structural, semantic, and type constraints established in the model to be satisfied.

State Space. The set of all admissible configurations is defined as

$$\mathcal{M}_{M_0} = \{\, M \mid \text{Inv}(M) \,\}.$$

Family of Transformations.

$$\mathcal{F}_{M_0} = \big\{\, f \mid f : \mathcal{M}_{M_0} \to \mathcal{M}_{M_0} \big\}.$$

Each element $f \in \mathcal{F}_{M_0}$ maps one admissible model configuration to another.

Basic Properties.

1. *Closure:* $M \in \mathcal{M}_{M_0} \;\Rightarrow\; f(M) \in \mathcal{M}_{M_0}$.
2. *Composition:* $f_2 \circ f_1 \in \mathcal{F}_{M_0}$ for $f_1, f_2 \in \mathcal{F}_{M_0}$.
3. *Identity:* $\text{id} \in \mathcal{F}_{M_0}$.

Thus, $(\mathcal{M}_{M_0}, \mathcal{F}_{M_0})$ forms a *monoid action*: composition defines an associative operation on $\mathcal{F}_{M_0}$, and the application of functions induces transitions between states $M \xrightarrow{f} M'$, $M, M' \in \mathcal{M}_{M_0}$.

10. Decision Nodes

Let $V_{\text{dec}} \subseteq V$ be the set of *decision nodes*. Each node $v \in V_{\text{dec}}$ is associated with a transformation.

$$f_v \in \mathcal{F}_{M_0}, \qquad M' = f_v(M).$$

Parameterization (Optional). The specific implementation of f_v may depend on the aggregated attributes $\mathrm{attr}^*(v)$ or other local data; formally, this does not affect the requirement that $f_v \in \mathcal{F}_{M_0}$.

Node Marking. Optionally, a node v can be marked with a special class $c_{\mathrm{dec}} \in C_V$ and/or equipped with flag attributes—their consistency is verified through the standard semantic morphism mechanism.

Activation. The mechanism for invoking f_v (e.g., event-driven, manual, or programmatic) is not specified; it is sufficient that the resulting state of the model remains within $\mathcal{M}_{M_0}$.

11. Hierarchy of Nested Levels of Model M_0

Let M_0 be the maximal set of elements and relations of the model. Subsets are defined within it that satisfy the following strict chain of inclusions:

$$M_{\mathrm{PK}} \subset M_{\mathrm{Sys}} \subset M_{\mathrm{LC}} \subset M_0.$$

- M_{LC}: The subgraph of the sequential phases of the system life cycle.
- M_{Sys}: The physical and functional configuration of the system.
- M_{PK}: The union of all start-up complexes of the system.

12. Phase Decomposition of the System Life Cycle

Phase Definition. The rule R_{LC} defines a non-empty, linearly ordered set of pairwise distinct elements

$$LC = \{\rho_1 < \rho_2 < \cdots < \rho_n\}, \qquad n \geq 2,$$

where each phase ρ_i corresponds to a logically complete state of the system.

Phase Subgraphs. A non-empty subgraph is associated with each phase:

$$G_{\rho_i} = (V_{\rho_i}, E_{\rho_i}) \subseteq M_{\mathrm{LC}}.$$

Partition Invariants

C1 (Complete Coverage).

$$\bigcup_{i=1}^{n} G_{\rho_i} = M_{\mathrm{LC}}.$$

All vertices and edges of the life cycle belong to at least one phase.

C2 (Local Overlap).

$$G_{\rho_i} \cap G_{\rho_j} \neq \varnothing \implies |i - j| \leq 1.$$

An element can belong to at most two adjacent phases.

Phase Transitions. For each pair of adjacent phases $\rho_i \to \rho_{i+1}$, a minimal (with respect to inclusion) transition set is defined:

$$Y_i^* \subseteq G_{\rho_i} \cap G_{\rho_{i+1}}, \qquad Y_i^* \neq \varnothing.$$

The transition occurs when the rule $R_{\text{tr}}\big(Y_i^*\big)$ is satisfied; its content is not specified here.

Thus, the life cycle is described by the sequence of subgraphs $G_{\rho_1}, \ldots, G_{\rho_n}$ within M_{LC}; the details of phase formation and transition conditions are encapsulated in the rules R_{LC} and R_{tr}.

The Life Cycle Model M_{LC}. By taking the union of the vertex and edge sets of all phase subgraphs, we obtain:

$$V_{\text{LC}} = \bigcup_{i=1}^{n} V_{\rho_i}, \qquad E_{\text{LC}} = \bigcup_{i=1}^{n} E_{\rho_i}.$$

Thus,

$$M_{\text{LC}} = (V_{\text{LC}}, E_{\text{LC}}) \subseteq M_0,$$

and it includes precisely those vertices and edges from the original model that participate in at least one phase of the life cycle.

Note on Life Cycle Phases. The transition from a phase ρ_i to an adjacent phase ρ_{i+1} of the life cycle can be implemented via a decision vertex $v \in V_{\text{dec}}$: invoking the corresponding function f_v transitions the model from subgraph G_{ρ_i} to $G_{\rho_{i+1}}$ while preserving all invariants of $\mathcal{M}_{M_0}$.

Note on the Semantic Content of Phases. Life cycle phases can encompass states in which the system does not yet have a physical or functional implementation. The specific semantic content of the phases, including physical, functional, or integration properties, is determined by the rules R_{LC}.

Note on Invariant C2. Non-adjacent phase intersections, such as $G_{\rho_i} \cap G_{\rho_{i+2}} \neq \varnothing$, are prohibited. Thus, C2 implements the principle of "minimal overlap": an element can only belong to the current and the immediately following (or preceding) phase. If a practical model requires the *extended presence of an element* across multiple phases, invariant C2 must be adjusted.

13. Representation of the System M_{Sys} on the Graph M_0

System Formalization Let the initial model be defined as

$$M_0 = \big(L,\ V,\ E_{\text{hor}},\ E_{\text{vert}},\ \theta\big).$$

Within the class hierarchy C_V, the following classes are identified:

$$c_{\text{phys}},\ c_{\text{func}} \in C_V,$$

where c_{phys} and c_{func} are special classes for physical and functional realization, respectively.

Note on Physical and Functional Realization. The class c_{phys} characterizes elements that have a physical realization, regardless of their ability to perform functions. The class c_{func} is assigned only to those elements that already physically exist and are capable of performing their functions. An element can be physically realized without being functionally active (e.g., equipment that is installed but not yet commissioned), but not vice versa.

$$V_{\text{Sys}} = \big\{\, v \in V \mid c_{\text{phys}} \in \kappa_V(v) \big\}.$$

For an edge e, let $\text{Ends}(e) \subseteq V$ denote the set of its incident vertices. Then,

$$E_{\text{Sys}} = \big\{\, e \in E_{\text{hor}} \cup E_{\text{vert}} \mid \text{Ends}(e) \subseteq V_{\text{Sys}} \big\}, \qquad M_{\text{Sys}} = \big(V_{\text{Sys}}, E_{\text{Sys}}\big).$$

- $c_{\text{func}} \in \kappa_V(v) \;\Rightarrow\; c_{\text{phys}} \in \kappa_V(v)$.

Connection with Life Cycle Phases M_{LC} Let $M_{\text{LC}} = \bigcup_{i=1}^{n} (V_{\rho_i}, E_{\rho_i})$ be the union of phases ρ_i. For any $v \in V_{\text{Sys}}$, there exists at least one phase ρ_i such that $v \in V_{\rho_i}$. The classes $c_{\text{phys}}, c_{\text{func}}$ can change only during the transitions Y_i^*.

Autonomous Subsystems (Startup Complexes)

$$G_{\text{PK}}^{(j)} = \big(V_{\text{PK}}^{(j)}, E_{\text{PK}}^{(j)}\big), \qquad j = 1, \ldots, m,$$

$$E_{\text{PK}}^{(j)} = \big\{ e \in E_{\text{Sys}} \mid \text{Ends}(e) \subseteq V_{\text{PK}}^{(j)} \big\}.$$

$$M_{\text{PK}} = \Big(\bigcup_j V_{\text{PK}}^{(j)}, \bigcup_j E_{\text{PK}}^{(j)} \Big) \subseteq M_{\text{Sys}}.$$

Shared elements:

$$V_{\text{shared}} = \bigcap_{j \neq k} \big(V_{\text{PK}}^{(j)} \cap V_{\text{PK}}^{(k)} \big), \qquad c_{\text{shared}} \in C_V.$$

Note on Shared Elements. V_{shared} represents components required for the operation of multiple startup complexes. Classic examples include infrastructure nodes, control systems, and auxiliary subsystems. Such elements are commissioned with the first startup complex and remain active for subsequent ones.

- **Root Vertex.** For each complex, a root vertex is fixed: $v_{\text{root}}^{(j)} = \arg\min_{v \in V_{\text{PK}}^{(j)}} \theta(v)$; if there are multiple such minimal vertices, one is chosen arbitrarily and used in all subsequent references.

  Note (Role of the Root Vertex). The root vertex aggregates the functionality of all its subordinate components; therefore, its readiness is interpreted as the readiness of the entire complex for operation.

- **Autonomy.** Any vertex $v \in V_{\text{PK}}^{(j)}$ is reachable from $v_{\text{root}}^{(j)}$ by a path that lies entirely within $G_{\text{PK}}^{(j)}$.

- **Functional Completeness.** There exists a morphism $\varphi \in \Phi$ (in Strict inheritance mode) such that the images $\{\varphi_V(v) \mid v \in V_{\text{PK}}^{(j)}\}$ realize all prescribed functions of the complex.

Attribute Model of Readiness Let

$$E_{\text{integ}} = \{\text{init}, \text{partial}, \ldots, \text{full}\}, \quad \text{Dom}_{\tau_{\text{integ}}} = E_{\text{integ}},\ \tau_{\text{func}} \in [0,1].$$

The attribute model provides a quantitative assessment of an element's readiness on the interval $[0,1]$, where 0 signifies a complete lack of readiness and 1 signifies full readiness. Reaching the value $\tau_{\text{func}} = 1$ automatically entails the assignment of class c_{func}, thus fixing the element's functional activity.

$$\alpha_{\min} \colon \{\text{func}(w) \mid w \in \mu_V(v)\} \longrightarrow \min(\,\cdot\,), \quad \tau_{\text{target}} = \tau_{\text{func}}.$$

- If $(\tau_{\text{func}}, 1) \in attr^*(v)$, then c_{func} is added to $\kappa_V(v)$.
- The complex $G_{\text{PK}}^{(j)}$ is considered ready when $(\tau_{\text{func}}, 1) \in attr^*\big(v_{\text{root}}^{(j)}\big)$.

Vertical Consistency

$$c_{\text{phys}} \in \kappa_V(v) \ \Rightarrow\ c_{\text{phys}} \in \kappa_V(w), \quad (v,w) \in E_{\text{vert}}.$$

$$c_{\text{func}} \in \kappa_V(v) \iff \Big(\forall w \in \mu_V(v) : c_{\text{func}} \in \kappa_V(w)\Big) \wedge \kappa'(w) \subseteq \kappa'(v).$$

The condition $\kappa'(w) \subseteq \kappa'(v)$ ensures that a parent vertex can be functionally active only when all its child elements are also active and their derived classifications are consistent with the parent's classification, thereby ensuring the integrity of the functional decomposition.

Final System Invariants The following invariants ensure the consistency of the system configuration throughout its entire life cycle and must be preserved under any permissible modification of the model.

- $V_{\text{Sys}} \neq \varnothing,\ V_{\text{PK}} = \bigcup_j V_{\text{PK}}^{(j)} \neq \varnothing.$
- Each vertex $v \in V_{\text{PK}}$ belongs to at least one complex $G_{\text{PK}}^{(j)}$.
- $c_{\text{func}} \in \kappa_V(v) \ \Rightarrow\ c_{\text{phys}} \in \kappa_V(v).$

14. Stage Filtration of the Startup Complex M_{PK}

Filtration of a Startup Complex For the subgraph $G_{PK}^{(j)} \subseteq M_{\text{Sys}}$, an ordered sequence of stages is defined:

$$\text{Stage}_0^{(j)} \subset \text{Stage}_1^{(j)} \subset \cdots \subset \text{Stage}_{m_j}^{(j)} = G_{PK}^{(j)}, \qquad \text{Stage}_k^{(j)} = \big(V_k^{(j)}, E_k^{(j)}\big).$$

The readiness metric for a stage is:

$$\text{Ready}_k^{(j)} \ = \min_{x \in V_k^{(j)} \cup E_k^{(j)}} \tau_{\text{func}}(x), \quad \tau_{\text{func}}(x) \in \text{attr}^*(x).$$

Invariants (for each k):

F-1 Root Reachability: $v_{\text{root}}^{(j)} \in V_k^{(j)}$, and there exists a directed path $v_{\text{root}}^{(j)} \rightsquigarrow v$ along the edges $E_k^{(j)} \cap E_{\text{vert}}$ for any $v \in V_k^{(j)}$.

F-2 Physical Completeness: $\forall v \in V_k^{(j)}\ (c_{\text{phys}} \in \kappa_V(v))$.

F-3 Strictly Increasing Readiness:

$$0 = \text{Ready}_0^{(j)} < \cdots < \text{Ready}_{m_j}^{(j)} = 1.$$

F-4 Vertical Closure: $(u, w) \in E_{\text{vert}},\ w \in V_k^{(j)} \ \Rightarrow\ u \in V_k^{(j)}$.

F-5 Minimality of the Step:

$$\nexists\, W \subsetneq \big(V_{k+1}^{(j)} \setminus V_k^{(j)}\big) : \qquad \min_{x \in V_k^{(j)} \cup W} \tau_{\text{func}}(x) \ \geq\ \text{Ready}_{k+1}^{(j)}.$$

Functional Content Let $E_0^{(j)} := \varnothing$. For a hyper-edge $e = (v_e, W_e) \in E_{\text{hor}}$, the vertex v_e is the *source*, and W_e are the *targets*.

$$\Gamma_k^{(j)} = \Big\{ e = (v_e, W_e) \in E_{\text{hor}} \ \Big|\ \{v_e\} \cup W_e \subseteq V_k^{(j)},\ e \notin E_{k-1}^{(j)} \Big\}, \qquad k \geq 1.$$

Stage Function For a vertex $v \in V_{m_j}^{(j)}$:

$$\text{stg}^{(j)}(v) \ =\ \min\{k \mid v \in V_k^{(j)}\}, \qquad (u, w) \in E_{\text{vert}} \ \Rightarrow\ \text{stg}^{(j)}(u) \ \leq\ \text{stg}^{(j)}(w).$$

Transition Predicate

$$\text{Trans}_k^{(j)} = \Big[\tau_{\text{func}}\big(v_{\text{root}}^{(j)}\big) \ \geq\ \text{Ready}_{k+1}^{(j)}\Big] \ \wedge\ \Big[\forall v \in V_{k+1}^{(j)} \setminus V_k^{(j)} : c_{\text{phys}} \in \kappa_V(v)\Big].$$

The transition $\text{Stage}_k^{(j)} \to \text{Stage}_{k+1}^{(j)}$ is established when $\text{Trans}_k^{(j)} = 1$.

Aggregated State

$$\text{stg}_{\text{curr}}^{(j)} = \max\big\{k \mid \text{Trans}_\ell^{(j)} = 1\ \ \forall \ell < k\big\}, \qquad \text{AggState} = \big(\text{stg}_{\text{curr}}^{(j)}\big)_{j \in J}.$$

The system is considered commissioned when

$$\min_{j \in J} \text{stg}_{\text{curr}}^{(j)} = m_j.$$

Connection with Life Cycle Phases

$$\vartheta\big(\text{Stage}_k^{(j)}\big) = \min\big\{\rho_i \mid \text{Stage}_k^{(j)} \subseteq M_{LC}^{(\rho_i)}\big\},$$

$$\xi(\rho) = \big\{\text{Stage}_k^{(j)} \mid \vartheta(\text{Stage}_k^{(j)}) = \rho\big\}, \qquad \xi : LC \to \mathcal{P}\Big(\big\{\text{Stage}_k^{(j)} \ \big|\ j \in J,\ 0 \leq k \leq m_j\big\}\Big).$$

Note on the Well-Definedness of ϑ and ξ For any stage $\text{Stage}_k^{(j)}$, there exists a phase $\rho_i \in LC$ such that $\text{Stage}_k^{(j)} \subseteq M_{LC}^{(\rho_i)}$; therefore, ϑ is always well-defined, and the composition $\xi \circ \vartheta$ recovers the original stage.

15. PERT and GERT Subgraphs

15.1 PERT Subgraphs

Definition 15.1 (PERT Subgraph)**.** Let $M = (V, E)$ be the base graph of the project. A subgraph $G_{\mathrm{PERT}} = (V_{\mathrm{p}}, E_{\mathrm{p}}) \subseteq (V, E)$ is called a *PERT subgraph* if the following conditions are met:

1. G_{PERT} is a directed acyclic graph (DAG);
2. Each edge $e \in E_{\mathrm{p}}$ is associated with a three-point estimate:
$$\mathrm{est}(e) = \big(\mathrm{time}_{\min}(e),\ \mathrm{time}_{\mathrm{mid}}(e)\ \text{(most-likely)},\ \mathrm{time}_{\max}(e)\big),$$
$$\mathrm{time}_{\min}(e) < \mathrm{time}_{\mathrm{mid}}(e) < \mathrm{time}_{\max}(e);$$
3. The pair of distribution parameters, the expected value $\mathbb{E}[\mathrm{time}_e]$ and the variance $\mathrm{Var}(\mathrm{time}_e)$, is determined by the transformation:
$$\big(\mathbb{E}[\mathrm{time}_e], \mathrm{Var}(\mathrm{time}_e)\big) = f_{\mathrm{P}}\big(\mathrm{est}(e)\big), \qquad f_{\mathrm{P}} \colon \mathbb{R}^3 \to \mathbb{R}^2_{>0};$$
4. The random duration of the edge time_e is described by the distribution:
$$\mathrm{time}_e \sim \mathcal{F}_{\mathrm{PERT}}\big(\mathbb{E}[\mathrm{time}_e], \mathrm{Var}(\mathrm{time}_e)\big),$$
where the set $\{\mathrm{time}_e\}_{e \in E_{\mathrm{p}}}$ is assumed to be pairwise independent unless a covariance matrix Cov is specified;
5. The critical path is defined by the condition of zero time slack for the vertices, where $t_E(v)$ and $t_L(v)$ are the earliest and latest event times for vertex v:
$$\mathrm{Slack}(v) = t_L(v) - t_E(v) = 0.$$

Note. The function f_{P} and the family of distributions $\mathcal{F}_{\mathrm{PERT}}$ are defined at the data level, which allows for the flexible use of both classical and modern parameterization schemes.

15.2 GERT Subgraph

A subgraph $\Gamma^{\mathrm{GERT}} = (V'', E'') \subseteq (V, E)$ is called a *GERT subgraph* if it represents a stochastic process model characterized by the following properties:

- Each edge $e = (v_i, v_j) \in E''$ is associated with a transition probability $p_{ij} \in [0, 1]$.
- For each branching node v_i, the normalization condition holds: $\sum_j p_{ij} = 1$.
- The graph structure allows for alternative paths and, optionally, controlled cycles (provided that their processing does not violate the global invariants of the model M).

The structure of a GERT subgraph can be a local fragment of the model or coincide with it:
$$\Gamma^{\mathrm{GERT}} \subseteq M \quad \text{or} \quad \Gamma^{\mathrm{GERT}} = M.$$

Analytical Purpose and Integration. GERT subgraphs are used to analyze subprocesses with non-deterministic outcomes. The analysis of such a subgraph allows for the computation of key probabilistic characteristics:

- The probability of realizing each of the possible final states.
- The distribution of final values for each path.

The results of this analysis are used for dynamic aggregation: the computed metrics serve to update the attributes of the parent (aggregating) vertex in the hierarchy of M.

Set of Scenarios. The collection of all valid realizations of stochastic processes in the model M is defined as the set of scenarios:

$$\Gamma = \left\{ \Gamma_i = (V_i, E_i) \,\middle|\, \Gamma_i \subseteq G_{\text{PERT}} \cup \Gamma^{\text{GERT}},\ \Gamma_i \subseteq M \right\},$$

where each scenario Γ_i represents one of the possible paths.

15.3 PDM Subgraph (Precedence Diagramming Method)

Optionally, within the structure $M = (L, V, E_{\text{hor}}, E_{\text{vert}}, \theta)$, a directed subgraph is defined:

$$\Gamma_{\text{PDM}} = (V_{\text{pdm}}, E_{\text{pdm}}) \subseteq (V, E_{\text{hor}}),$$

satisfying the conditions:

$$V_{\text{pdm}} \neq \varnothing, \qquad E_{\text{pdm}} \subseteq \{(u, v) \mid u, v \in V_{\text{pdm}}\}.$$

Relationship Types A finite subset is defined:

$$C_E^{\text{PDM}} = \{\text{FS}, \text{SS}, \text{FF}, \text{SF}\} \subseteq C_E$$

and for any edge $e \in E_{\text{pdm}}$, the following holds:

$$\kappa_E(e) \in C_E^{\text{PDM}}, \qquad |\kappa_E(e)| = 1.$$

Lag Attribute If $e \in E_{\text{pdm}}$, then an attribute $(\tau_{\text{lag}}, \ell) \in \text{attr}_E(e)$ with a value $\ell \in \mathbb{Z}$ is permitted.

Invariants

1. The subgraph Γ_{PDM} is a directed acyclic graph.
2. The type correctness of all attributes is inherited from the system defined previously.

Note The calculation of time metrics (ES/EF/LS/LF, float (slack)) and the derivation of derived criticality classes are performed in an external layer Q and are not specified in this section.

16. Matrix Representation of Model M for ML/AI Applications

Preliminary Transformation of the Level Graph For each level $L \in \mathsf{Levels}(M)$, the initial horizontal hypergraph (V_L, E_L^{hor}) is reduced to a directed graph $G'(L) = (V_L, E'_L)$ (a multigraph is allowed):

$$(v, W) \in E_L^{\text{hor}} \longmapsto \{(v, w) \mid w \in W\} \subseteq E'_L.$$

Duplicate edges are aggregated by summing their weights. Let $n_L = |V_L|$ and $m_L = |E'_L|$.

Set of Level Matrices $\mathcal{M}(L)$ The following set of matrices is defined on the graph $G'(L)$:

$$\mathcal{M}(L) = \{A_L,\ B_L,\ D_L,\ L_L,\ \mathcal{L}_L,\ P_L,\ A_L^k,\ T_L,\ R_L\}, \qquad k \in \mathbb{N},$$

divided into three logical groups.

Fundamental Matrices

A_L **Adjacency Matrix** $\big(A_L[i,j]\big) \in \mathbb{N}_0^{n_L \times n_L}$: $A_L[i,j]$ is the sum of weights of the edges from v_i to v_j.

B_L **Incidence Matrix** $\big(B_L[i,e]\big) \in \{-1,0,1\}^{n_L \times m_L}$: $B_L[i,e] = 1$ if v_i is the source of edge e, $B_L[i,e] = -1$ if v_i is the sink, and 0 otherwise.

Derived Structural Matrices

D_L **Diagonal Degree Matrix** $D_L = \operatorname{diag}\big(\deg^+ v_1, \ldots, \deg^+ v_{n_L}\big) \in \mathbb{N}_0^{n_L \times n_L}$, where $\deg^+$ is the out-degree of a vertex.

L_L **Unnormalized Laplacian** $L_L = D_L - A_L$ (also known as the "Kirchhoff matrix").

$\mathcal{L}_L$ **Normalized Symmetric Laplacian** $\mathcal{L}_L = D_L^{-1/2} L_L D_L^{-1/2}$.

P_L **Stochastic Transition Matrix** $P_L = D_L^{-1} A_L$.

Analytical Matrices

A_L^k **k-step path matrix**: element $[i,j]$ is the number of paths of length k from v_i to v_j.

T_L **Reachability Matrix**: $T_L[i,j] = 1$ if a path exists from v_i to v_j, and 0 otherwise.

R_L **Distance Matrix**: $R_L[i,j]$ is the length of the shortest path from v_i to v_j; for unreachable pairs, $R_L[i,j] = \infty$.

The set $\mathcal{M}(L)$ provides a complete numerical representation of level L and serves as a basis for further graph-based analysis and learning procedures.

17. Global Matrix Representation of the Model M

By ordering the hierarchy levels $L_1, \ldots, L_H$ and concatenating all vertices into a single list $\big[V(L_1) \,\|\, V(L_2) \,\|\, \ldots \,\|\, V(L_H)\big]$, we construct the block adjacency matrix

$$A_{\text{glob}} = \begin{pmatrix} A^{(1)} & A^{(1,2)} & 0 & \cdots \\ 0 & A^{(2)} & A^{(2,3)} & \cdots \\ \vdots & \ddots & \ddots & \ddots \end{pmatrix},$$

where $A^{(h)}$ denotes the connections *within* level L_h, while $A^{(h,h+1)}$ denotes the connections *between* neighbouring levels. This matrix completely determines the topology of the entire model.

Practical storage formats

- **Dense array**. A conventional two-dimensional array; applicable only to very small graphs (requires $\mathcal{O}(N^2)$ memory).
- **Classical sparse formats**. Only non-zero elements are stored ($\mathcal{O}(m)$ memory instead of $\mathcal{O}(N^2)$); the typical choice for large graphs.
- **Block-sparse format**. Each non-empty block $A^{(h)}$ or $A^{(h,h+1)}$ is stored separately, which can be efficient for hierarchical structures.
- **Multi-channel representation**. Horizontal and vertical edges are stored as separate "channels" of a single object, allowing independent weights to be assigned during subsequent graph-based training.

Note. The formats listed above are merely examples of possible implementations; particular schemes can be refined, extended, or otherwise adapted to specific requirements, tasks, and tools.

18. Semantic Layer Model S

The semantic layer is defined by the tuple

$$S = \big(T,\ \text{Rel},\ R;\ W,\ \ell;\ C,\ \gamma;\ \alpha_T,\ \alpha_R;\ sym,\ tr,\ acyc\big),$$

where

- T — a finite set of terms;
- Rel — a finite set of semantic relation types;
- $R \subseteq T \times \text{Rel} \times T$ — a set of directed triplets (t_1, λ, t_2);
- W — a set of word forms and lexemes;
- $\ell : T \to 2^W$ — a mapping from terms to sets of lexical forms;
- C — a finite set of term categories (e.g., `Activity`, `Milestone`);

- $\gamma : T \to C$ — the term categorization function;
- $\alpha_T : T \to \mathcal{P}(A_T),\ \alpha_R : R \to \mathcal{P}(A_R)$ — functions that associate a term or a relation with a *set* of attributes of the form

$$\langle \texttt{key} : \text{String},\ \texttt{type} : \tau \in AType,\ \texttt{value} : \text{Dom}_\tau,\ \texttt{source} : \text{String}\rangle;$$

  for a given object, keys are unique and are replaced if repeated;
- $sym,\ tr,\ acyc : \text{Rel} \to \{0, 1\}$ — meta-tags of relation types (1 means the property is present, 0 means absent).

Integrity invariants

1. **Local acyclicity.** If $acyc(\lambda) = 1$, then the subgraph $R_\lambda = \{(t_1, \lambda, t_2) \in R\}$ contains no cycles.
2. **Attribute type correctness.** For any record $\langle \texttt{key}, \texttt{type}, \texttt{value}, \texttt{source}\rangle \in \alpha_T(t)$ ($t \in T$) or $\in \alpha_R(r)$ ($r \in R$), it holds that $\texttt{value} \in \text{Dom}_{\texttt{type}}$.

Minimal set of semantic relation types

Type	Description	*sym*	*tr*	*acyc*
`phaseOf`	membership in a phase	0	1	1
`partOf`	part–whole relation	0	1	1
`covers`	covering generalization (non-transitive)	0	0	1
`precedes`	logical/temporal precedence	0	1	1
`relatedTo`	symmetric association	1	0	0

Note. By default, $tr(\texttt{covers}) = 0$ to avoid the "avalanche" of inferences that can arise when different kinds of generalizations are mixed in a model. If the model is already carefully normalized and a classical transitive *is-a* hierarchy is required, it suffices to set $tr(\texttt{covers}) = 1$.
Standards note. When helpful, it is recommended that these relations be aligned with widely used vocabularies—W3C RDFS/OWL/SKOS/OWL-Time, OGC GeoSPARQL, OBO RO/BFO, W3C PROV-O, and DCMI Terms—as reference points for modeling.

Scope (implementation-neutral). S may be implemented either as a lightweight dictionary of terms and relations or as a full onto-linguistic domain model (ontology+lexicon). Both options are consistent with §18 and the core invariants; moreover, S is intentionally extensible and may be substantially extended without affecting the base structure.

On the relation between S and classifications. The semantic layer S and the classification system κ_V, κ_E with hierarchies H_V, H_E (§§4–5) are distinct mechanisms. This paper allows multiple interaction policies between them—from tight coupling ("hard links", e.g., projecting S relations into M via (ψ, ω), §19) to loose coupling or full separation ("soft links" where S provides only annotations/validation). In all cases, H_V, H_E remain the authoritative structures for formal class constraints, while S complements them with semantic relations, terminology, and, when useful, a mirrored taxonomy via `subclassOf`/`instanceOf` (see above).

Recommended add-on relation types for the semantic layer S (model-local defaults).

Type	**Description**	**sym**	**tr**	**acyc**	**Note**
`subclassOf`	Taxonomy: genus–species (class → superclass).	0	1	1	Partial order over classes; inherit axioms/constraints, not property values.
`instanceOf`	Typing: individual belongs to a class.	0	0	1	Directed "individual → class"; non-transitive.
`partOf`	Meronymy: part–whole (component composition).	0	1	1	*Component* meronymy only; do not confuse with `memberOf`; no value inheritance.
`memberOf`	Membership in a collection/group.	0	0	0	Not meronymy; handle group nesting by explicit policy if needed.
`locatedIn`	Spatial inclusion / containment.	0	1	1	Set tr=1, $acyc$=1 *only* when areas are nested within a single spatial model.
`adjacentTo`	Spatial adjacency / boundary contact.	1	0	0	Symmetric, irreflexive; no transitivity.
`before`	Temporal precedence (strict).	0	1	1	Strict partial order (asymmetric, irreflexive).
`preconditionOf`	Precondition required by an action/event.	0	0	1	Non-transitive by default; keep acyclic in practice.
`participatesIn`	Entity participates in an event/process.	0	0	1	Model as a bipartite fragment (entity ↔ event).
`hasRole`	Plays a thematic role (agent/patient/tool).	0	0	1	Roles are realized within concrete events/processes.
`hasProperty`	Has an attribute/characteristic.	0	0	1	Tie to typed attributes; not transitive.
`derivedFrom`	Provenance / derivation chain.	0	1	1	Directed; chainable; typically acyclic.
`causes`	Causal relation.	0	1	1	Use tr=1 only if your causal reasoning policy requires; otherwise set tr=0.
`sameAs`	Identity / equivalence.	1	1	0	Strong equivalence; use with care.
`differentFrom`	Explicit non-identity / incompatibility.	1	0	0	Symmetric and irreflexive; useful for contradiction checks.

Policy: $sym, tr, acyc \in \{0, 1\}$ are model-local defaults for validation and may be overridden per project. This add-on set complements the minimal S-types (`phaseOf`, `partOf`, `covers`, `precedes`, `relatedTo`) defined in the model.

Remark (cross-level identity). Vertices on different levels linked by E_{vert} remain distinct in M. Possible identity of such vertices at the domain level is expressed in S (e.g. *sameAs*, ψ/ω) and may be mirrored by classifications (shared class in κ_V or identical ϕ_V); structural invariants of M are unaffected; cf. §§4–5, §19.

Hard vs soft linking (optional). The interaction between S and class hierarchies H_V, H_E (§§4–5) can be configured: *hard links* (projecting S relations into M via (ψ, ω), §19) or *soft links* (annotation/validation only). Class constraints remain governed by H_V, H_E regardless of the policy.

Semantic-network view (optional). Layer S may be defined/implemented as a semantic network: a directed, labeled graph $G_S = (T, E_S)$ with vertex set T and edge set $E_S \equiv R \subseteq T \times \mathit{Rel} \times T$. In standard mathematical terms, a graph is a pair (V, E); under the usual logical reading adopted here, vertices denote elements of the domain, each relation type $\lambda \in \mathit{Rel}$ is a binary predicate symbol, and every triple $(t_1, \lambda, t_2) \in R$ asserts the ground atom $\lambda(t_1, t_2)$. This view is optional, compatible with §19's bidirectional links, and can be extended as needed without redefining the classification hierarchies.

N-ary relations via reification (optional). Some domain facts are inherently n-ary, $\lambda(a_1, \ldots, a_n)$. We represent such a fact by introducing a dedicated event/fact node $r \in T$ (category $\gamma(r) = \mathit{Event}$) and linking each participant a_i to r with a binary semantic relation `hasRole`; the role name is stored as a typed attribute `role` $\in \mathrm{Enum}(R_\lambda)$ on that triple via α_R (§18). Evidence from the corpus D is attached to r and/or its links using semantic pointers (policy thresholds may apply; §§20–21).

The bidirectional link (ψ, ω) aligns S and M (§19): r projects to a structural vertex v_r, and for each participant $u_i \in \omega(a_i)$ we materialize a same-level horizontal edge $e_i = (v_r, \{u_i\})$ with $\kappa_E(e_i) =$ `hasRole` and the `role` attribute on the edge. To preserve intra-level acyclicity, inverse links like `participatesIn` are typically derived on demand rather than materialized together. Minimal checks: (i) required roles present (cardinalities by policy); (ii) level locality for all e_i; (iii) Inv-HorAcyc holds (§§2–3); (iv) S-consistency of edge types (§19).

Example. HydrostaticTest with roles `agent`=*CrewA*, `object`=*Loop#1*, `tool`=*Pump P-101*; optionally, `precedes`(*HydrostaticTest*, *LoopFlushing*) with `acyc`=1 (§18).

Multilingual lexicon (optional). To support multiple languages, extend the lexical map to $\ell_{\text{lng}} : T \to 2^{W \times \text{Lang}}$ with optional preferred labels $\text{pref} : T \times \text{Lang} \to W$. Multiple forms per language (synonyms, abbreviations) are allowed; they remain lexical views of the same term $t \in T$ and do not introduce new terms. This addition does not affect class hierarchies or invariants of §18.

Domain/range constraints for relations (optional). For each $\lambda \in \text{Rel}$ one may specify domain and range sets $C_{\text{dom}}(\lambda), C_{\text{rng}}(\lambda) \subseteq C$. Validation requires, for every triple $(t_1, \lambda, t_2) \in R$, that $\gamma(t_1) \in C_{\text{dom}}(\lambda)$ and $\gamma(t_2) \in C_{\text{rng}}(\lambda)$. When checking edges in M against R (§19), the project may use a universal ($\forall$) or existential ($\exists$) policy; both preserve the base invariants.

Term identifiers and namespaces (optional). Introduce a stable identifier $\mathrm{id}_T : T \to \Sigma^* \cup \{\perp\}$; uniqueness is enforced on the pair $(\mathrm{ns}, \mathrm{id}_T)$. Human-readable names and translations live in ℓ/ℓ_{lng} and may change without altering id_T. This mechanism is internal to layer S and does not modify H_V, H_E or the structural invariants.

19. Semantic Enrichment of the Graph $\mathcal{M}$

The integration of the structural graph $\mathcal{M} = (V, E)$ with the semantic layer S is defined by a pair of mutually inverse mappings

$$\psi : V \cup E \longrightarrow \mathcal{P}(T), \qquad \omega : T \longrightarrow \mathcal{P}(V \cup E),$$

that satisfy

$$t \in \psi(x) \iff x \in \omega(t).$$

A fixed classification function

$$\kappa_E : E \longrightarrow \mathrm{Rel}$$

assigns to every edge its semantic relation type from layer S.

Semantic effects. The bidirectional pair (ψ, ω) enables

- clustering of graph elements according to the term categories $\gamma : T \to C$;
- aggregation of subsets of $V \cup E$ annotated with inter-related terms;
- logical inference of additional edges and annotations via the relation set $R \subseteq T \times \mathrm{Rel} \times T$;
- traceability of terminology changes through the mapping ω;
- alignment with external ontologies by means of the lexical mapping $\ell : T \to 2^W$.

Purpose of the bidirectional link

1. *Projection $M \to S$.* The image ψ assigns terminological classes to graph elements, allowing the rules of layer S to be applied to $\mathcal{M}$.
2. *Projection $S \to M$.* The image ω yields the set of structural instances associated with a given term, thereby ensuring traceability.

Formal invariants

1. **Mutual consistency.** $t \in \psi(x) \;\Leftrightarrow\; x \in \omega(t)$.
2. **Totality and bounded multiplicity.** There exists a parameter $k_{\max} \in \mathbb{N}$ (recommended $1 \le k_{\max} \le 5$ for nuclear-plant engineering models) such that $\forall x \in V \cup E : 1 \le |\psi(x)| \le k_{\max}$.
3. **Categorical compatibility.**

$$v \in V \wedge t \in \psi(v) \;\Rightarrow\; \gamma(t) \in C_V, \qquad e \in E \wedge t \in \psi(e) \;\Rightarrow\; \gamma(t) \in C_E.$$

4. **Consistency with layer-S relations.** Let $e = (v_1, v_2) \in E$ and $\lambda = \kappa_E(e)$. Then
$$\forall t_1 \in \psi(v_1),\ t_2 \in \psi(v_2) :\ (t_1, \lambda, t_2) \in R.$$
5. **Attribute inheritance.** When keys coincide, the later operand has priority:
$$\alpha(x) = \operatorname*{fold}_{t \in \psi(x)} \operatorname{merge}\big(\alpha_0(x),\, \alpha_T(t)\big), \qquad \beta(t) = \operatorname*{fold}_{x \in \omega(t)} \operatorname{merge}\big(\varnothing,\, \alpha_0(x)\big).$$

Functional implications

1. **Clustering.** Grouping of graph elements by ψ and γ.
2. **Impact analysis.** For a term t, the set $\omega(t)$ specifies all affected structural elements.
3. **Auto-completion.** Applying the rules of S generates additional elements in ψ and ω, while preserving invariants B1–B5.

Synchronization algorithm

Init. Build ψ_0 from structural templates and derive ω_0 via B1.

Update$_M$. Upon modification of an element x, recompute $\psi(x)$ and then refresh ω.

Update$_S$. When a new term t' is added, compute candidate elements x by the matching rule and subsequently update ψ.

Consistency Check. Periodically verify B1–B5; violations trigger a rollback of the transaction.

20. Corpus $\mathcal{D}$ and the System of Semantic Pointers

The corpus is defined as a countable set of documents.
$$\mathcal{D} \;=\; \{\, d_i \mid i \in \mathbb{N} \,\},$$
and for each $d \in \mathcal{D}$ the following are fixed: a unique identifier $\mathrm{id}(d) \in \mathbb{N}$, an abstract type $\mathrm{type}(d)$, and a timestamp $\mathrm{ts}(d)$.

The set of semantic pointers is
$$\mathcal{P} \;=\; \big\{ p = (d, \mathrm{loc}, \mathrm{meta}) \mid d \in \mathcal{D},\ \mathrm{loc} \in \mathrm{Loc}(d),\ \mathrm{meta} \in \mathrm{Meta}(d) \big\},$$
where $\mathrm{Loc}(d)$ and $\mathrm{Meta}(d)$ are abstract domains specified individually for each document d.

21. Mappings between M, S, and $\mathcal{D}$

Let $M = (V, E)$ be the model graph and T the set of terms of the semantic layer S. Three mappings are defined:
$$\phi : V \cup E \;\longrightarrow\; 2^T, \qquad \omega : T \;\longrightarrow\; 2^{V \cup E}, \qquad \psi : T \;\longrightarrow\; 2^{\mathcal{P}},$$
linked by the axiom
$$x \in \omega(t) \iff t \in \phi(x) \tag{$\mathbf{A_1}$}$$
and the connectivity condition
$$\operatorname{im} \phi \;\subseteq\; \operatorname{dom} \psi.$$

Evidence set. For $x \in V \cup E$ define

$$\mathrm{Evidence}(x) \;=\; \bigcup_{t \in \phi(x)} \psi(t).$$

21.1 Integrity invariants

1. **Source existence:** $\forall p \in \mathcal{P}\ (p.d \in \mathcal{D})$.
2. **Localization validity:** $\forall p \in \mathcal{P}\ (p.\mathrm{loc} \in \mathrm{Loc}(p.d))$.
3. **Content immutability:** any change to the content yields a new element $d' \neq d$ with $\mathrm{id}(d') \neq \mathrm{id}(d)$.

Target invariant (*well-formed*). A configuration is considered correctly documented iff

$$\forall t \in T\ \ \psi(t) \neq \varnothing \quad \Longleftrightarrow \quad \forall t \in T\ \ \omega(t) \neq \varnothing.$$

Basic coverage requirement

$$\kappa_0(x) \;=\; \big(\mathrm{Evidence}(x) \neq \varnothing\big), \qquad x \in V \cup E.$$

The model is said to be *documented* if $\kappa_0(x)$ holds for every $x \in V \cup E$.

Projection operation (sub-corpus) For an arbitrary predicate $P : \mathcal{D} \to \{0,1\}$ set

$$\mathcal{D} \restriction P = \{\, d \in \mathcal{D} \mid P(d) = 1 \,\}, \quad \mathcal{P} \restriction P = \{\, p \in \mathcal{P} \mid p.d \in \mathcal{D} \restriction P \,\}.$$

The mappings ϕ, ω, ψ are naturally restricted to $(\mathcal{D} \restriction P, \mathcal{P} \restriction P)$, and invariants **I1**–**I3** together with axiom $\mathbf{A_1}$ are preserved without additional conditions.

21.2 Optional extensions

- A numerical coverage measure $\kappa(x) = |\,\mathrm{Evidence}(x)|$ and a data-quality function q_D.
- A document-version graph, access-control policies, multilingual mappings, and extended semantic relations R.
- Categorical constraints of the form $\gamma(t) \in C_V$ (for vertices) and $(t_1, \lambda, t_2) \in R$ (for edges), introduced in a separate module.

Each extension must be accompanied by a check that invariants **I1**–**I3** and axiom $\mathbf{A_1}$ remain satisfied.

22. Classifications of the Corpus $\mathcal{D}$

The corpus $\mathcal{D}$ is organised by a set of independent classifications, each of which describes the documents from a particular perspective—topical, typological, status-related, and so on. Each classification is represented as a tree, providing multi-level categorisation, efficient retrieval, and the formation of arbitrary sub-corpora. The number of such classifications is unbounded; they do not interfere with one another and may evolve independently.

Formal scheme. Let I be a countable index set. For every $i \in I$ a hierarchy

$$H_i = (C_i, \preceq_i),$$

is given, where C_i is a finite set of classes and $\preceq_i$ is a partial order forming a tree (the root is the "most general" class, the leaves are specialised ones). Documents are annotated by a total function

$$\kappa_i : \mathcal{D} \longrightarrow C_i, \qquad i \in I,$$

so that each $d \in \mathcal{D}$ is assigned exactly one class in every hierarchy.

Key properties.

1. **Hierarchical.** Every H_i is a tree without cycles, since $\preceq_i$ is transitive, antisymmetric, and directed from the root to the leaves.
2. **Multiplicity.** Different indices i are independent; a document can be classified along several orthogonal criteria without conflicts.
3. **Compositionality.** For any predicate P on $\mathcal{D}$, the projection $\mathcal{D} \restriction P$ induces sub-trees $H_i \restriction P$, preserving the tree structure and the totality of κ_i.

Trigger (Class-Sync). Any change in κ_i automatically initiates the re-validation of the morphisms φ and the associated mappings ψ, ω.

Relation to the semantic layer. When required, the classes C_i can be linked to the terms T of the semantic layer S via mappings $\sigma_i : C_i \to 2^T$, which enables the automatic inference of classification features, for example, on the basis of pointer contents drawn from $\mathcal{P}$. Detailed rules for such integration are specified in domain-oriented extensions and are not involved in the core corpus invariants.

23. Prefix-Hierarchical Coding Systems and a Functorial Extension of the Model

Fix an abstract *prefix-hierarchical positional coding system* $\mathcal{C}$. The industry-standard KKS satisfies all the axioms stated below and is treated as the canonical representative of $\mathcal{C}$, yet the model remains applicable to any scheme that fulfills the same properties.

Axioms

1. **Injectivity.** There exists a mapping $\mathrm{id}_{\mathcal{C}} : V \to \Sigma^*$ that assigns to every $v \in V$ a unique code c.
2. **Block decomposition.** For every code c a finite ordered list of blocks is defined: $\mathrm{blk}(c) = \langle b_1, \ldots, b_m \rangle$.
3. **Prefix hierarchy.** On V introduce the relation $u \sqsubset v \iff \mathrm{blk}(u)$ is a strict prefix of $\mathrm{blk}(v)$; hence $(V, \sqsubseteq)$ is a partially ordered set.
4. **Positional semantics.** A function $\sigma : \mathbb{N} \times \Sigma^* \to C$ ("position $\to$ class") assigns a semantic class to every block.

The functor $F_{\mathcal{C}} : M \to M^{\mathcal{C}}$ The functor acts identically on the cardinalities of the underlying sets but rewrites auxiliary functions as computations over the code:

$$\begin{aligned}
&v \mapsto c = \mathrm{id}_{\mathcal{C}}(v), && \theta_{\mathcal{C}}(v) = |\,\mathrm{blk}(c)|,\\
&(u,v) \in E_{\mathrm{vert}} \iff c_u \sqsubset c_v, && \kappa_V(v) = \{\sigma(i,b_i)\}_{i=1}^{|\,\mathrm{blk}(c)|},\\
&feval(u,v) = \big(1 + \mathrm{LCP}(c_u,c_v)\big)^{-1}, && \Sigma_{\mathcal{C}} :\ \text{roll-up by common prefix.}
\end{aligned}$$

23.1 Additional invariants of layer $M^{\mathcal{C}}$

- **(C1) Prefix-Vertical.** $(u,v) \in E_{\mathrm{vert}} \iff c_u \sqsubset c_v$.
- **(C2) Level-Consistency.** $\theta_{\mathcal{C}}(u) = \theta_{\mathcal{C}}(v) \iff |\,\mathrm{blk}(c_u)| = |\,\mathrm{blk}(c_v)|$.
- **(C3) Class-Totality.** $\forall v \in V :\ \kappa_V(v) \neq \varnothing$, since σ is defined for every position.

All invariants C1–C3 strengthen, but do not modify, the original constraints of the model M, and are therefore compatible with the material of Sections 1–6.

Generalisation and reversibility The functor $F_{\mathcal{C}}$ affects only the semantics of identifiers; "de-injection" is achieved by simply discarding the string representation, leaving the sets V, E, and the basic axioms intact. Hence

1. the model is applicable even without coding;
2. any other coding scheme that satisfies Axioms A1–A4 can be attached through a similar functor without changes to the main text;
3. mixed scenarios are admissible, because invariants C1–C3 are checked only for vertices that possess a code c.

With a prefix-hierarchical coding in place, the basic model gains an automatically computable hierarchy, classes, and aggregates, which significantly simplifies invariant checking and increases formal rigor.

24. Set of Metrics $\mathcal{Q}$

Let $\mathcal{X} = \{\mathcal{M}, \mathcal{S}, \mathcal{D}\}$ be the basic layers of the model, and let T denote a discrete or continuous time domain.

Definition 24.1 (Metric)**.** Every metric $q \in \mathcal{Q}$ is defined on a non-empty subset $\mathcal{X}' \subseteq \mathcal{X}$ and depends on time:

$$q : \Big(\prod_{X_k \in \mathcal{X}'} X_k\Big) \times T \longrightarrow \mathbb{R} \cup \mathbb{R}^k \cup [0,1].$$

The set of metrics is written as an indexed family $\mathcal{Q} = \{q_i\}_{i \in I_{\mathcal{Q}}}$ and is functionally bound to a specific configuration $(\mathcal{M}, \mathcal{S}, \mathcal{D})$.

24.1 Classification by Domain of Definition

The classification of $\mathcal{Q}$ is inherited from the domain on which each metric is defined:

$$\mathcal{Q} = \mathcal{Q}^{(\mathcal{M})} \cup \mathcal{Q}^{(\mathcal{S})} \cup \mathcal{Q}^{(\mathcal{D})} \cup \mathcal{Q}^{(\mathcal{M},\mathcal{S})} \cup \mathcal{Q}^{(\mathcal{S},\mathcal{D})} \cup \mathcal{Q}^{(\mathcal{M},\mathcal{S},\mathcal{D})},$$

where membership $q \in \mathcal{Q}^{(\cdot)}$ is determined by the components of $\mathcal{X}$ that appear in the domain of q.

24.2 Relation to the Attribute System

Let A be the set of typed attributes introduced in Section 6. The mapping

$$q_{\text{attr}} : A \longrightarrow \mathcal{Q}$$

fixes those attributes that are interpreted as metrics, preserving type consistency and preventing semantic duplication.

24.3 Algebraic Structure of Layer $\mathcal{Q}$

$$\big(\mathcal{Q},\ \circledast,\ \uplus,\ \varsigma\big),$$

- $\circledast : \mathcal{Q}\times\mathcal{Q} \to \mathcal{Q}$ — composition (weighted aggregation);
- $\uplus : \mathcal{Q}\times\mathcal{Q} \to \mathcal{Q}$ — parallel union (vectorisation);
- $\varsigma : \mathcal{Q} \to \mathcal{Q}$ — unary transformation (normalization, scaling).

$$q_1, q_2 \in [0,1] \implies q_1 \circledast q_2 \in [0,1].$$

Type preservation. For any $q_1, q_2 \in \mathcal{Q}$ with codomains V_1, V_2, the result $q_1 \circledast q_2$ has a codomain V that is consistent with V_1 and V_2 according to the type-composition rules.

Commutative monoid. The operations $\circledast$ and $\uplus$ make $\big(\mathcal{Q}, \circledast, \uplus\big)$ a commutative monoid with the neutral element $q_\perp$.

24.4 Partial Order and Optimality

For scalar metrics, a natural partial order is defined:

$$q_1 \preceq q_2 \quad :\Longleftrightarrow \quad \forall (m,s,d) \in \mathcal{M}\times\mathcal{S}\times\mathcal{D}\ \forall t \in T:\ q_1(m,s,d,t) \le q_2(m,s,d,t).$$

The relation $\preceq$ is used in dominance analysis and optimization (Pareto front).

24.5 Projection Operators and the Locality Principle

- **Projection.** For $X \in \{\mathcal{M},\mathcal{S},\mathcal{D}\}$ an operator $\pi_X : \mathcal{Q}^{(\mathcal{M},\mathcal{S},\mathcal{D})} \to \mathcal{Q}^{(X)}$ extracts the local component of a metric.
- **Locality.** If $\phi : \mathcal{M}_1 \to \mathcal{M}_2$ is a structural isomorphism, then $q(\mathcal{M}_1,\mathcal{S},\mathcal{D},t) = q\big(\phi(\mathcal{M}_1),\mathcal{S},\mathcal{D},t\big)$ for every $q \in \mathcal{Q}^{(\mathcal{M})}$ and all $t \in T$.

24.6 Interface with the Meta-Level

$$\iota : \mathcal{Q} \longrightarrow U,$$

where U is the layer of strategic artifacts of the model (see Section 24.6).

Resulting Structure of Layer $\mathcal{Q}$

$$\big(\mathcal{Q},\ \{\mathcal{Q}^{(\cdot)}\},\ \circledast, \uplus, \varsigma,\ \preceq,\ q_{\text{attr}},\ \pi_X,\ \iota\big)\ \dashv\ (\mathcal{M}, \mathcal{S}, \mathcal{D}, T).$$

25. Meta-Level of the Model U

25.1 Basic Configuration $\mathcal{U}_0$

Definition 25.1 (Meta-level without ML extension)**.**

$$\mathcal{U}_0 = (\mathcal{X},\ \mathcal{F},\ \Omega), \qquad \mathcal{X} = \{M,\ S,\ D,\ Q\},$$

where

M a multi-level structural model;

S the semantic layer of terms and relations;

D an external corpus of data and documents;

Q the layer of quantitative metrics and statistical functions;

$\mathcal{F}$ a set of mappings $f_{ik} : X_i \to X_k,\ X_i, X_k \in \mathcal{X}$;

Ω a set of *meta-operations* $\omega : \mathcal{X} \to \mathcal{X}$.

Consistency invariant. Let $\mathcal{X}_{\text{valid}} \subseteq \mathcal{X}$ be the set of valid configurations. For any $\omega \in \Omega$ it holds that

$$\forall\, X \in \mathcal{X}_{\text{valid}} \quad \Longrightarrow \quad \omega(X) \in \mathcal{X}_{\text{valid}},$$

i.e. the result of every permissible operation remains valid. Specifically for the metric layer this implies $\omega(Q) \subseteq Q \ \wedge\ \text{Inv}_Q\big(\omega(Q)\big)$, where Inv_Q denotes the meta-type correctness predicate (non-negativity, dimensional consistency, etc.).

25.2 Extended Meta-Level with ML/AI Support

Definition 25.2 (ML extension)**.**

$$\mathcal{U}_{\text{ML}} = \big(\mathcal{X},\ \mathcal{F},\ \Omega,\ \mathcal{ML}\big),$$

where $\mathcal{X}, \mathcal{F}, \Omega$ coincides with the basic configuration, and the additional component

$$\mathcal{ML} = \big(\mathcal{D}_{\text{train}},\ \mathcal{A},\ \Pi,\ \mathcal{Z}\big)$$

specifies the machine-learning processes:

- $\mathcal{D}_{\text{train}} \subseteq \mathcal{P}(M \cup S \cup D \cup Q)$ — training datasets;
- $\mathcal{A}$ — a set of algorithms and models;
- $\Pi = \{\pi_k : \mathcal{D}_k \to \mathcal{D}_{k+1}\}$ — an iterative data-refinement strategy;
- $\mathcal{Z} \subseteq \mathcal{P}(M \cup S \cup D \cup Q) \times \mathcal{Y} \times \mathcal{Q}$ — a suite of tests/validation benchmarks.

Note. There is an embedding $\iota : \mathcal{U}_0 \hookrightarrow \mathcal{U}_{\mathrm{ML}}$ preserving the composition of meta-operations $\circ : \Omega \times \Omega \to \Omega$, so that $(\Omega, \circ)$ forms a monoid and ι is a monoid homomorphism. This property is treated as a *model assumption* and must be verified for any concrete realization of $\circ$.

25.3 Classification of Meta-Operations

The set Ω decomposes into the disjoint union

$$\Omega = \Omega_{\mathrm{stat}} \sqcup \Omega_{\mathrm{gen}} \sqcup \Omega_{\mathrm{train}} \sqcup \Omega_{\mathrm{eval}},$$

where Ω_{stat} — aggregation/filtering, Ω_{gen} — generation of new elements, Ω_{train} — model training, Ω_{eval} — metric evaluation and logging. A typical cycle

$$\Omega_{\mathrm{gen}} \;\to\; \Omega_{\mathrm{train}} \;\to\; \Omega_{\mathrm{eval}} \;\xrightarrow{\text{filter by } Q}\; \Omega_{\mathrm{gen}}$$

guarantees monotonic improvement of a preferred metric $\mu \in Q$.

25.4 Adaptive Generation of Elements

A function

$$\Upsilon : (\mathcal{X}(t), \mathcal{Q}(t)) \longrightarrow \mathcal{P}(M) \times \mathcal{P}(S) \times \mathcal{P}(D) \times \mathcal{P}(Q)$$

produces generation domains $M^{\mathrm{gen}}(t), S^{\mathrm{gen}}(t), D^{\mathrm{gen}}(t), Q^{\mathrm{gen}}(t)$. For any x in these domains there exists $\omega_t \in \Omega_{\mathrm{gen}}$ such that $\omega_t(l) \mapsto x,\ l \in L$. $\mathsf{VALIDATE}(x)$ checks:

1. syntactic correctness;
2. semantic annotation by an AI morphism $f^{\mathrm{AI}} \in F_{\mathrm{AI}}$;
3. satisfaction of error thresholds $\varepsilon(x) \le \varepsilon_{\max}$;
4. consistent integration into M, S, D, or Q.

25.5 Categorical Interpretation of the Mappings

The mappings $\mathcal{F}$ form a small category

$$\mathbf{Cat}_F = \big\langle \mathrm{Ob} = \{M, S, D, Q\},\ \mathrm{Hom} = F \big\rangle.$$

An AI morphism f^{AI} is accompanied by a fine-tuning operator $\partial f^{\mathrm{AI}} \in \Omega_{\mathrm{train}}$.

Assumption (functoriality of fine-tuning). For the sake of formal tractability, we assume

$$\partial(g \circ f) = \partial g \circ \partial f.$$

This condition must be verified for every concrete ML pipeline.

Summary

The meta-level U forms a formal control loop that orchestrates components M, S, D, Q. The extension $\mathcal{U}_{\mathrm{ML}}$ introduces learning and evaluation, while the cluster Ω realizes the cycle *generation* $\to$ *training* $\to$ *evaluation* $\to$ *adaptation*, maintaining model correctness and monotonically improving the key metrics.

Glossary

High-Level Model Components, Unions, and Notational Conventions

The model rests on five fundamental components. For each of them, a **Latin uppercase letter** denotes its *basic description*, whereas the **calligraphic variant** denotes the corresponding *generalisation*: a space, class, or family of such descriptions. In addition, the symbol X is introduced to express an *aggregated union* of several components.

M / $\mathcal{M}$ — **structural model** a multi-level directed graph that defines the structure and topology;

S / $\mathcal{S}$ — **semantic layer** a terminological and lexico-semantic representation aligned with the structural model;

D / $\mathcal{D}$ — **Corpus** an aggregated store of external data of arbitrary nature, integrated with the model via links and annotations;

Q / $\mathcal{Q}$ — **quantitative layer** an abstract representation of measurable characteristics in the form of metrics and statistical functions;

U / $\mathcal{U}$ — **meta-level** a coordination and orchestration loop that unites the components M, S, D, Q and specifies the admissible operations on them;

X / $\mathcal{X}$ — **aggregated union** a chosen subset of components, e.g. $X = \{M, S, D\}$, while $\mathcal{X}$ denotes the space of all admissible unions.

Mappings between the Basic Components

X the basic set of model components $\{M, S, D, Q\}$;

F the collection of *all* mappings (functions, morphisms) between elements of X or their internal structures;

Ω the set of *meta-operations*, i.e. procedures that modify, generate, or compose functions from F;

F_{stat} the subset of *statically* defined (non-trainable) mappings;

F_{AI} the subset of *trainable* (AI/ML-based) mappings, $F_{\text{AI}} \subseteq F$;

f a reserved symbol for an *arbitrary* mapping between two components or their parts, $f : X_i \to X_j,\ X_i, X_j \in X$;

f_{XY} a named function *from* component X *to* Y (e.g., $f_{MD} : M \to D$);

$f', f^{(k)}$ variations and versions of a function f;

$\text{dom}(f), \text{im}(f)$ the domain and image of a mapping (standard notation);

Conventions

1. When additional semantics of a mapping need to be clarified, a short explanatory tag may be appended in the index, e.g. $f_{\text{ann}} : M \to S$ for annotations or $f_{\text{sync}} : D \to M$ for data synchronisation.

2. If the same symbol f is reused in different contexts, its meaning must be specified *at first mention* (either in the formula caption or in the immediate text).

3. The letter Ω does not fix an implementation method; concrete semantics (function composition, training, validation, etc.) are given in the algorithm description or in the "Meta-Level" section.

Basic Notation Reserved for M_0

The symbols below are reserved for the basic structure M_0. Each symbol's meaning is stated briefly; formal definitions are provided in the main body of the model.

L an ordered set of levels;

V the set of vertices;

E_{hor} the set of horizontal hyper-edges within a level;

E_{vert} the set of vertical edges between adjacent levels;

E the union of all edges, $E_{\text{hor}} \cup E_{\text{vert}}$;

G the complete graph (V, E);

Y the union of all structural elements, $V \cup E_{\text{hor}} \cup E_{\text{vert}}$;

θ the "vertex level" mapping, $\theta : V \to L$;

μ_V the vertex-aggregation mapping;

μ_E the edge-aggregation mapping (reserved, unused in M_0);

δ the level-refinement mapping (reserved, unused in M_0);

l a level index ($l \in L$);

v, w individual vertices ($v, w \in V$);

e an individual edge ($e \in E$);

W a non-empty subset of V acting as the targets of a hyper-edge;

$\mathcal{P}^+(V)$ the set of all non-empty subsets of V;

$V^{(l)}$ the subset of vertices at level l;

$E^{(l)}$ the subset of horizontal edges at level l;

$G^{(l)}$ the horizontal sub-graph of level l, $(V^{(l)}, E^{(l)})$;

$\mathrm{UG}^{(l)}$ the undirected version of $G^{(l)}$;

$\mathrm{Reach}_l^+(v)$ the set of vertices at level l reachable from v via directed hyper-paths of non-zero length;

$\mathrm{Reach}_l^*(v)$ the set $\mathrm{Reach}_l^+(v) \cup \{v\}$;

m, n integer counters (lengths, indices);

k the maximum level index, $k = \max L$.

Classifications and the Semantic Morphism

C_V the set of vertex classes; $\preceq_V$ — a partial order on C_V;

C_E the set of edge classes; $\preceq_E$ — a partial order on C_E;

c_{vert} a special class of vertical edges, $c_{\mathrm{vert}} \in C_E$; for every $e \in E_{\mathrm{vert}}$ we have $c_{\mathrm{vert}} \in \kappa_E(e)$;

κ_V the vertex-classification mapping: $\kappa_V : V \to \mathcal{P}(C_V)$ (each vertex is assigned a set of admissible classes);

κ_E the edge-classification mapping: $\kappa_E : E \to \mathcal{P}(C_E)$ (each edge is assigned a set of admissible classes);

φ_V the vertex component of the semantic morphism: $\varphi_V : V \to C_V$;

φ_E the edge component of the semantic morphism: $\varphi_E : E \to C_E$;

φ the full semantic morphism, $\varphi = (\varphi_V, \varphi_E)$;

Φ the set of all admissible semantic morphisms φ;

$\mathrm{depth}_V(c)$ the depth of a class $c \in C_V$ in the hierarchy $(C_V, \preceq_V)$, $\mathrm{depth}_V : C_V \to \mathbb{N}_{\geq 0}$;

f_{eval} an evaluation function for semantic morphisms: $f_{\mathrm{eval}} : \Phi \to \mathbb{R}_{\geq 0}$ (the concrete specification depends on the task).

Typed Attributes

VType the set of basic value types (e.g., $\mathbb{R}$, $\mathbb{Z}$, Bool, $\mathrm{Cat}(C), \ldots$);

Val_v the universe of values for a basic type $v \in \mathrm{VType}$;

AType the set of applied attribute types;

β a mapping that links an applied attribute type to a basic type: $\beta : \mathrm{AType} \to \mathrm{VType}$;

Dom_τ the value domain of an applied type $\tau \in \mathrm{AType}$, where $\mathrm{Dom}_\tau \subseteq \mathrm{Val}_{\beta(\tau)}$;

A the universe of correctly typed attributes: $A = \{\, (\tau, \mathrm{val}) \mid \tau \in \mathrm{AType},\ \mathrm{val} \in \mathrm{Dom}_\tau \,\}$;

attr_V the vertex-attribute mapping, $\mathrm{attr}_V : V \to \mathcal{P}(A)$;

attr_E the edge-attribute mapping, $\mathrm{attr}_E : E \to \mathcal{P}(A)$;

attr^* extended (inherited / aggregated) attributes of an element (the precise definition is given later in the text).

Attributes, Aggregates, and Generalisations

attr^* extended attributes of an element;

Λ the set of aggregate functions;

$\alpha \in \Lambda$ an aggregate function;

$\mathrm{InBase}(\alpha)$ the basic input type of α;

$\mathrm{ResType}(\alpha)$ the basic result type of α;

$\mathrm{AcceptedTypes}(\alpha)$ (opt.) the admissible applied input types of α;

$\mathcal{A}(\cdot)$ the operator that joins attributes of a set of elements: $\mathcal{A} : \mathcal{P}(V \cup E) \to \mathcal{P}(A)$, $\mathcal{A}(S) = \bigcup_{e \in S} \mathrm{attr}(e)$;

$\mathrm{Input}_{\alpha_r}(\mathrm{elem})$ the input set for function α_r in rule r; see the formula in the model text;

Σ_{agg} the set of aggregation rules;

$r \in \Sigma_{\mathrm{agg}}$ a rule $r = (\tau^{(r)}_{\mathrm{target}},\ \mathrm{ContextSelector}_r,\ \mathrm{AttributeSelector}_r,\ \alpha_r,\ \mathrm{default_val}_r)$;

$\tau^{(r)}_{\mathbf{target}}$ the applied type of the attribute created by rule r;

α_r the aggregate function used in rule r;

$\mathrm{val}^{(r)}_{\mathbf{target}}$ the computed value of the aggregated attribute;

C' the set of derived (generalised) classes;

κ' the derived classification, $\kappa' \colon V \to \mathcal{P}(C')$;

$\Pr(c, v, \mathrm{attr}^*(v))$ the predicate "vertex v belongs to class c" based on extended attributes;

`SemInheritanceMode` the inheritance mode, `STRICT` or `LOOSE`;

h an applied function that uses attr^*;

$\mathrm{ContextSelector}_r$ $\mathrm{ContextSelector}_r : V \cup E \to \mathcal{P}(V \cup E)$;

$\mathrm{AttributeSelector}_r$ $\mathrm{AttributeSelector}_r : \mathcal{P}(A) \times \Lambda \to \mathcal{P}(A)$;

$\mathrm{default_val}_r$ the default value for rule r, $\mathrm{default_val}_r \in \mathrm{Val}_{\mathrm{ResType}(\alpha_r)}$.

Iterative Processes

Γ_{iter} the set of iterative components;

γ an individual component from Γ_{iter};

v_{entry}, v_{exit} the entry and exit vertices of γ;

P_{loop} a directed hyper-path forming the loop;

P_{bypass} a horizontal hyper-edge forming the bypass;

$N_{\max}$ the maximum number of steps in P_{loop};

Cond a logical predicate selecting the path;

Vertices(γ) an operator returning the set of all vertices in component γ.

Model Nesting Hierarchy

M_0 the maximal set of vertices V and edges E of the original model;

$M_{LC} \subseteq M_0$ the sub-graph that describes the sequential life-cycle phases;

$M_{\mathbf{Sys}} \subseteq M_{LC}$ the sub-graph that fixes the physical and functional system configuration.

Phase Decomposition of the Life-Cycle (M_{LC})

$LC = \{\rho_1 < \rho_2 < \cdots < \rho_n\}$ a non-empty linearly ordered set of phases ($n \geq 2$);

ρ_i the i-th life-cycle phase, corresponding to a logically completed state;

$G_{\rho_i} = (V_{\rho_i}, E_{\rho_i}) \subseteq M_{LC}$ a non-empty sub-graph associated with phase ρ_i;

$Y_i^* \subseteq G_{\rho_i} \cap G_{\rho_{i+1}}$ the minimal (by inclusion) non-empty transition set of elements for the transition $\rho_i \to \rho_{i+1}$;

$V_{LC} = \bigcup_{i=1}^{n} V_{\rho_i}$ the set of all vertices that participate in at least one phase;

$E_{LC} = \bigcup_{i=1}^{n} E_{\rho_i}$ the set of all edges that participate in at least one phase;

R_{LC} a rule that defines the composition and order of phases LC;

$R_{\mathbf{tr}}$ a rule (binary relation) of admissible transitions between phases.

References

[1] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, Oct. 2023. DOI: 10.48550/arXiv.2310.11511.

[2] A. Banerjee and A. Char, *On the spectrum of directed uniform and non-uniform hypergraphs*, Aug. 2018. DOI: 10.48550/arXiv.1710.06367.

[3] C. Berge, *Hypergraphs: combinatorics of finite sets* (North-Holland mathematical library v. 45). Amsterdam: North Holland [u.a.], 1989, ISBN: 978-0-444-87489-4.

[4] W. Bounsi *et al.*, *Transformers meet Neural Algorithmic Reasoners*, Jun. 2024. DOI: 10.48550/arXiv.2406.09308.

[5] A. Bretto, *Hypergraph Theory: An Introduction* (Mathematical Engineering). Heidelberg: Springer, 2013, ISBN: 978-3-319-00079-4. DOI: 10.1007/978-3-319-00080-0.

[6] T. B. Brown *et al.*, *Language Models are Few-Shot Learners*, May 2020. DOI: 10.48550/arXiv.2005.14165.

[7] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, *Spectral Networks and Locally Connected Networks on Graphs*, Dec. 2013. DOI: 10.48550/arXiv.1312.6203.

[8] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A Survey on Mixture of Experts in Large Language Models," *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–20, Jul. 2025. DOI: 10.1109/TKDE.2025.3554028.

[9] J. H. Caufield *et al.*, "Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): A method for populating knowledge bases using zero-shot learning," *Bioinformatics*, vol. 40, no. 3, J. Wren, Ed., btae104, Mar. 2024. DOI: 10.1093/bioinformatics/btae104.

[10] J. Chen, P. Hu, E. Jimenez-Ruiz, O. M. Holter, D. Antonyrajah, and I. Horrocks, "OWL2Vec*: Embedding of OWL ontologies," *Machine Learning*, vol. 110, no. 7, pp. 1813–1845, Jul. 2021. DOI: 10.1007/s10994-021-05997-6.

[11] J. Chen, K. Gao, G. Li, and K. He, *NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs*, Jun. 2022. DOI: 10.48550/arXiv.2206.04910.

[12] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, *Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks*, May 2019. DOI: 10.1145/3292500.3330925.

[13] J. Cooper and A. Dutle, "Spectra of uniform hypergraphs," *Linear Algebra and its Applications*, vol. 436, no. 9, pp. 3268–3292, May 2012. DOI: 10.1016/j.laa.2011.11.018.

[14] S. J. Cox, "Time ontology extended for non-Gregorian calendar applications," *Semantic Web*, vol. 7, no. 2, M. Gahegan, Ed., pp. 201–209, Feb. 2016. DOI: 10.3233/SW-150187.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Oct. 2018. DOI: 10.48550/arXiv.1810.04805.

[16] K. Ding, J. Wang, J. Li, D. Li, and H. Liu, "Be More with Less: Hypergraph Attention Networks for Inductive Text Classification," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2020, pp. 4927–4936. DOI: 10.18653/v1/2020.emnlp-main.399.

[17] V. P. e.V, *KKS Identification System for Power Stations — Guideline for Application and Key Part (VGB-S-811-01-2018-01-EN), 8th Edition; incl. Amendment G11/2019 (as of 2022-12)*. Essen: VGB PowerTech Service GmbH, 2018, ISBN: 978-3-96284-045-7.

[18] D. Edge *et al.*, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, Apr. 2024. DOI: 10.48550/arXiv.2404.16130.

[19] A. Estornell and Y. Liu, "Multi-LLM Debate: Framework, Principals, and Interventions," Nov. 2024. [Online]. Available: https://openreview.net/forum?id=sy7eSEXdPC&nesting=2&sort=date-desc.

[20] A. E. Ezugwu *et al.*, *Classical Machine Learning: Seventy Years of Algorithmic Learning Evolution*, Aug. 2024. DOI: 10.48550/arXiv.2408.01747.

[21] W. Fan *et al.*, *Graph Machine Learning in the Era of Large Language Models (LLMs)*, Apr. 2024. DOI: 10.48550/arXiv.2404.14928.

[22] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph Neural Networks," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 3558–3565, Jul. 2019. DOI: 10.1609/aaai.v33i01.33013558.

[23] Z. Feng, R. Wang, T. Wang, M. Song, S. Wu, and S. He, *A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges*, May 2024. DOI: 10.48550/arXiv.2405.00476.

[24] G. Gallo, G. Longo, S. Pallottino, and S. Nguyen, "Directed hypergraphs and applications," *Discrete Applied Mathematics*, vol. 42, no. 2-3, pp. 177–201, Apr. 1993. DOI: 10.1016/0166-218X(93)90045-P.

[25] F. Galuppi, R. Mulas, and L. Venturello, "Spectral theory of weighted hypergraphs via tensors," *Linear and Multilinear Algebra*, vol. 71, no. 3, pp. 317–347, Feb. 2023. DOI: 10.1080/03081087.2022.2030659.

[26] Y. Gao *et al.*, *Retrieval-Augmented Generation for Large Language Models: A Survey*, Dec. 2023. DOI: 10.48550/arXiv.2312.10997.

[27] Y. Ge *et al.*, *OpenAGI: When LLM Meets Domain Experts*, Apr. 2023. DOI: 10.48550/arXiv.2304.04370.

[28] A. Grattafiori *et al.*, *The Llama 3 Herd of Models*, Jul. 2024. DOI: 10.48550/arXiv.2407.21783.

[29] J. Gu *et al.*, *A Survey on LLM-as-a-Judge*, Mar. 2025. DOI: 10.48550/arXiv.2411.15594.

[30] Y. Gu, H. You, J. Cao, M. Yu, H. Fan, and S. Qian, *Large Language Models for Constructing and Optimizing Machine Learning Workflows: A Survey*, Nov. 2024. DOI: 10.48550/arXiv.2411.10478.

[31] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, *REALM: Retrieval-Augmented Language Model Pre-Training*, 2020. DOI: 10.48550/arXiv.2002.08909.

[32] W. L. Hamilton, R. Ying, and J. Leskovec, *Inductive Representation Learning on Large Graphs*, Jun. 2017. DOI: 10.48550/arXiv.1706.02216.

[33] J. Hoffmann *et al.*, *Training Compute-Optimal Large Language Models*, Mar. 2022. DOI: 10.48550/arXiv.2203.15556.

[34] S. Hong *et al.*, *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*, Aug. 2023. DOI: 10.48550/arXiv.2308.00352.

[35] E. J. Hu *et al.*, *LoRA: Low-Rank Adaptation of Large Language Models*, Jun. 2021. DOI: 10.48550/arXiv.2106.09685.

[36] IAEA, *Configuration Management in Nuclear Power Plants (IAEA-TECDOC-1335)*. Vienna: International Atomic Energy Agency, 2003, ISBN: 92-0-100503-2.

[37] IAEA, *Application of Configuration Management in Nuclear Power Plants*. Vienna: International Atomic Energy Agency, 2010, ISBN: 978-92-0-106710-4.

[38] IAEA, *Project Management in Nuclear Power Plant Construction: Guidelines and Experience (NP-T-2.7)*. Vienna: International Atomic Energy Agency, 2012, ISBN: 978-92-0-122210-7.

[39] IAEA, *Commissioning Guidelines for Nuclear Power Plants (NP-T-2.10)*. Vienna: International Atomic Energy Agency, 2018, ISBN: 978-92-0-102816-7.

[40] IAEA, *Economic Assessment of the Long Term Operation of Nuclear Power Plants (NP-T-3.25)*. Vienna: International Atomic Energy Agency, 2018, ISBN: 978-92-0-104218-7.

[41] IAEA, *Application of Plant Information Models to Manage Design Knowledge through the Nuclear Power Plant Life Cycle (IAEA-TECDOC-1919)*. Vienna: International Atomic Energy Agency, 2020, ISBN: 978-92-0-111120-3.

[42] IAEA, *Approaches to Management of Requirement Specifications for Nuclear Facilities throughout Their Life Cycle (IAEA-TECDOC-1933)*. Vienna: International Atomic Energy Agency, 2020, ISBN: 978-92-0-126120-5.

[43] IAEA, *Management of Nuclear Power Plant Projects (NG-T-1.6)*. Vienna: International Atomic Energy Agency, 2020, ISBN: 978-92-0-104719-9.

[44] IAEA, *Integrated Life Cycle Risk Management for New Nuclear Power Plants (NR-T-2.15)*. Vienna: International Atomic Energy Agency, 2023, ISBN: 978-92-0-101423-8.

[45] IAEA, *Considerations for Deploying Artificial Intelligence Applications in the Nuclear Power Industry (NR-T-1.26)*. Vienna: International Atomic Energy Agency, 2025, ISBN: 978-92-0-115525-2.

[46] J. Kaplan *et al.*, *Scaling Laws for Neural Language Models*, Jan. 2020. DOI: 10.48550/arXiv.2001.08361.

[47] A. Kau, X. He, A. Nambissan, A. Astudillo, H. Yin, and A. Aryani, *Combining Knowledge Graphs and Large Language Models*, arXiv:2407.06564 [cs], Jul. 2024. DOI: 10.48550/arXiv.2407.06564.

[48] S. D. Kim, R. K. Hammond, and J. E. Bickel, "Improved Mean and Variance Estimating Formulas for PERT Analyses," *IEEE Transactions on Engineering Management*, vol. 61, no. 2, pp. 362–369, May 2014. DOI: 10.1109/TEM.2014.2304977.

[49] S. Kim, S. Y. Lee, Y. Gao, A. Antelmi, M. Polato, and K. Shin, *A Survey on Hypergraph Neural Networks: An In-Depth and Step-By-Step Guide*, arXiv:2404.01039 [cs], Apr. 2024. DOI: 10.1145/3637528.3671457.

[50] D. Kreuzer, D. Beaini, W. L. Hamilton, V. Létourneau, and P. Tossou, *Rethinking Graph Transformers with Spectral Attention*, Jun. 2021. DOI: 10.48550/arXiv.2106.03893.

[51] P. Lewis *et al.*, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, May 2020. DOI: 10.48550/arXiv.2005.11401.

[52] Y. Li *et al.*, *Improving Multi-Agent Debate with Sparse Communication Topology*, Jun. 2024. DOI: 10.48550/arXiv.2406.11776.

[53] G. Liu, Y. Zhang, Y. Li, and Q. Yao, *Dual Reasoning: A GNN-LLM Collaborative Framework for Knowledge Graph Question Answering*, Jun. 2024. DOI: 10.48550/arXiv.2406.01145.

[54] S. Liu, C. Gao, and Y. Li, *Large Language Model Agent for Hyper-Parameter Optimization*, Feb. 2024. DOI: 10.48550/arXiv.2402.01881.

[55] Y. Liu *et al.*, *RoBERTa: A Robustly Optimized BERT Pretraining Approach*, Jul. 2019. DOI: 10.48550/arXiv.1907.11692.

[56] Z. Liu *et al.*, *OntoTune: Ontology-Driven Self-training for Aligning Large Language Models*, Feb. 2025. DOI: 10.48550/arXiv.2502.05478.

[57] A. Lo, A. Q. Jiang, W. Li, and M. Jamnik, *End-to-End Ontology Learning with Large Language Models*, Oct. 2024. DOI: 10.48550/arXiv.2410.23584.

[58] D. Luo, C. Feng, Y. Nong, and Y. Shen, *AutoM3L: An Automated Multimodal Machine Learning Framework with Large Language Models*, Aug. 2024. DOI: 10.48550/arXiv.2408.00665.

[59] D. G. Malcolm, J. H. Roseboom, C. E. Clark, and W. Fazar, "Application of a Technique for Research and Development Program Evaluation," *Operations Research*, vol. 7, no. 5, pp. 646–669, Oct. 1959. DOI: 10.1287/opre.7.5.646.

[60] C. Mavromatis and G. Karypis, *GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning*, May 2024. DOI: 10.48550/arXiv.2405.20139.

[61] S. Minaee *et al.*, *Large Language Models: A Survey*, Feb. 2024. DOI: 10.48550/arXiv.2402.06196.

[62] NASA, Ed., *NASA Systems engineering handbook. NASA/SP-2016-6105 Rev 2* (NASA SP). Washington, D.C: National Aeronautics and Space Administration (Office of the Chief Engineer), 2016, ISBN: 978-1-68092-090-1. [Online]. Available: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf.

[63] A. J. Oche, A. G. Folashade, T. Ghosal, and A. Biswas, *A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions*, Jul. 2025. DOI: 10.48550/arXiv.2507.18910.

[64] I. Ong *et al.*, *RouteLLM: Learning to Route LLMs with Preference Data*, Jun. 2024. DOI: 10.48550/arXiv.2406.18665.

[65] L. Ouyang *et al.*, *Training language models to follow instructions with human feedback*, Mar. 2022. DOI: 10.48550/arXiv.2203.02155.

[66] B. Peng *et al.*, *Graph Retrieval-Augmented Generation: A Survey*, Aug. 2024. DOI: 10.48550/arXiv.2408.08921.

[67] A. A. B. Pritsker, "GERT: Graphical Evaluation and Review Technique," Tech. Rep., Jan. 1966. [Online]. Available: https://www.rand.org/pubs/research_memoranda/RM4973.html.

[68] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, May 2023. DOI: 10.48550/arXiv.2305.18290.

[69] C. Raffel *et al.*, *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, Oct. 2019. DOI: 10.48550/arXiv.1910.10683.

[70] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, *Recipe for a General, Powerful, Scalable Graph Transformer*, May 2022. DOI: 10.48550/arXiv.2205.12454.

[71] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, *Temporal Graph Networks for Deep Learning on Dynamic Graphs*, Oct. 2020. DOI: 10.48550/arXiv.2006.10637.

[72] L. v. Rueden *et al.*, *Informed Machine Learning - A Taxonomy and Survey of Integrating Knowledge into Learning Systems*, Mar. 2019. DOI: 10.1109/TKDE.2021.3079836.

[73] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*, Feb. 2024. DOI: 10.48550/arXiv.2402.07927.

[74] I. H. Sarker, "Machine Learning: Algorithms, Real-World Applications and Research Directions," *SN Computer Science*, vol. 2, no. 3, p. 160, May 2021. DOI: 10.1007/s42979-021-00592-x.

[75] T. Schick *et al.*, *Toolformer: Language Models Can Teach Themselves to Use Tools*, Feb. 2023. DOI: 10.48550/arXiv.2302.04761.

[76] S. Schulhoff *et al.*, *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*, Jun. 2024. DOI: 10.48550/arXiv.2406.06608.

[77] K. Sharma, P. Kumar, and Y. Li, *OG-RAG: Ontology-Grounded Retrieval-Augmented Generation For Large Language Models*, 2024. DOI: 10.48550/arXiv.2412.15235.

[78] A. Shehzad *et al.*, *Graph Transformers: A Survey*, Jul. 2024. DOI: 10.48550/arXiv.2407.09777.

[79] H. Shirzad, A. Velingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, *Exphormer: Sparse Transformers for Graphs*, Mar. 2023. DOI: 10.48550/arXiv.2303.06147.

[80] S. Somvanshi, M. M. Islam, M. S. Mimi, S. B. B. Polock, G. Chhetri, and S. Das, *From S4 to Mamba: A Comprehensive Survey on Structured State Space Models*, May 2025. DOI: 10.48550/arXiv.2503.18970.

[81] O. Subasi, O. Bel, J. Manzano, and K. Barker, *The Landscape of Modern Machine Learning: A Review of Machine, Distributed and Federated Learning*, Dec. 2023. DOI: 10.48550/arXiv.2312.03120.

[82] M. Sun *et al.*, *A Survey on Large Language Model-based Agents for Statistics and Data Science*, Dec. 2024. DOI: 10.48550/arXiv.2412.14222.

[83] T. Talaei Khoei and N. Kaabouch, "Machine Learning: Models, Challenges, and Research Directions," *Future Internet*, vol. 15, no. 10, p. 332, Oct. 2023. DOI: 10.3390/fi15100332.

[84] T. Talaei Khoei, H. Ould Slimane, and N. Kaabouch, "Deep learning: Systematic review, models, challenges, and research directions," *Neural Computing and Applications*, vol. 35, no. 31, pp. 23 103–23 124, Nov. 2023. DOI: 10.1007/s00521-023-08957-4.

[85] S. Toro *et al.*, "Dynamic Retrieval Augmented Generation of Ontologies using Artificial Intelligence (DRAGON-AI)," *Journal of Biomedical Semantics*, vol. 15, no. 1, p. 19, Oct. 2024. DOI: 10.1186/s13326-024-00320-3.

[86] H. Touvron *et al.*, *LLaMA: Open and Efficient Foundation Language Models*, Feb. 2023. DOI: 10.48550/arXiv.2302.13971.

[87] P. Trirat, W. Jeong, and S. J. Hwang, *AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML*, Oct. 2024. DOI: 10.48550/arXiv.2410.02958.

[88] Y.-D. Tsai, Y.-C. Tsai, B.-W. Huang, C.-P. Yang, and S.-D. Lin, *AutoML-GPT: Large Language Model for AutoML*, Sep. 2023. DOI: 10.48550/arXiv.2309.01125.

[89] A. Vaswani *et al.*, *Attention Is All You Need*, Jun. 2017. DOI: 10.48550/arXiv.1706.03762.

[90] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, *Graph Attention Networks*, Oct. 2017. DOI: 10.48550/arXiv.1710.10903.

[91] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, *Deep Graph Infomax*, Sep. 2018. DOI: 10.48550/arXiv.1809.10341.

[92] VGB PowerTech e.V, *KKS Application Explanations. Parts A, B1–B4 (VGB-B 106e)*. Essen: VGB PowerTech Service GmbH, 2004, ISBN: 978-3-86875-831-3.

[93] VGB PowerTech e.V, *RDS-PP — Reference Designation System for Power Plants. Letter Codes for Basic Functions and Product Classes (VGB-B 102)*. Essen: VGB PowerTech Service GmbH, 2010, ISBN: 978-3-86875-352-3.

[94] VGB PowerTech e.V, *RDS-PP — Reference Designation System for Power Plants. Letter Code for Power Plant Systems (System Key) (VGB-S-821-00-2016-06-EN)*. Essen: VGB PowerTech Service GmbH, 2016, ISBN: 978-3-86875-945-7.

[95] W3C, *SKOS Simple Knowledge Organization System Reference*, 2009. [Online]. Available: https://www.w3.org/TR/skos-reference.

[96] W3C, *OWL 2 Web Ontology Language Document Overview (Second Edition)*, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/.

[97] W3C, *PROV-O: The PROV Ontology*, 2013. [Online]. Available: https://www.w3.org/TR/prov-o.

[98] W3C, *RDF 1.2 Schema*, 2025. [Online]. Available: https://www.w3.org/TR/rdf12-schema/.

[99] D. D. Walden and International Council on Systems Engineering, Eds., *Systems engineering handbook: a guide for system life cycle processes and activities. (INCOSE-TP-2003–002-05)*, Fifth edition. Hoboken, NJ: Wiley, 2023, ISBN: 978-1-119-81430-6.

[100] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, *Mixture-of-Agents Enhances Large Language Model Capabilities*, Jun. 2024. DOI: 10.48550/arXiv.2406.04692.

[101] L. Wang *et al.*, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186 345, 2023. DOI: 10.1007/s11704-024-40231-1.

[102] X. Wang *et al.*, *Executable Code Actions Elicit Better LLM Agents*, Feb. 2024. DOI: 10.48550/arXiv.2402.01030.

[103] J. Wei *et al.*, *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, Jan. 2022. DOI: 10.48550/arXiv.2201.11903.

[104] J. D. Wiest, "Precedence diagramming method: Some unusual characteristics and their implications for project managers," *Journal of Operations Management*, vol. 1, no. 3, pp. 121–130, Feb. 1981. DOI: 10.1016/0272-6963(81)90015-2.

[105] Q. Wu *et al.*, *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*, Aug. 2023. DOI: 10.48550/arXiv.2308.08155.

[106] Q. Wu, W. Zhao, Z. Li, D. Wipf, and J. Yan, *NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification*, Jun. 2023. DOI: 10.48550/arXiv.2306.08385.

[107] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2019. DOI: 10.1109/TNNLS.2020.2978386.

[108] H. Xia *et al.*, *Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding*, Jun. 2024. DOI: 10.48550/arXiv.2401.07851.

[109] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, May 2023. DOI: 10.48550/arXiv.2305.18323.

[110] J. Xu *et al.*, *Large Language Models Synergize with Automated Machine Learning*, May 2024. DOI: 10.48550/arXiv.2405.03727.

[111] M. Yang and X.-J. Xu, *Recent Advances in Hypergraph Neural Networks*, Mar. 2025. DOI: 10.48550/arXiv.2503.07959.

[112] S. Yao *et al.*, *ReAct: Synergizing Reasoning and Acting in Language Models*, Oct. 2022. DOI: 10.48550/arXiv.2210.03629.

[113] S. Yao *et al.*, *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, May 2023. DOI: 10.48550/arXiv.2305.10601.

[114] S. Yin *et al.*, "A Survey on Multimodal Large Language Models," *National Science Review*, vol. 11, no. 12, nwae403, Nov. 2024, arXiv:2306.13549 [cs]. DOI: 10.1093/nsr/nwae403.

[115] C. Ying *et al.*, *Do Transformers Really Perform Bad for Graph Representation?* Jun. 2021. DOI: 10.48550/arXiv.2106.05234.

[116] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, *Hierarchical Graph Representation Learning with Differentiable Pooling*, Jun. 2018. DOI: 10.48550/arXiv.1806.08804.

[117] Z. Yu, L. He, Z. Wu, X. Dai, and J. Chen, *Towards Better Chain-of-Thought Prompting Strategies: A Survey*, Oct. 2023. DOI: 10.48550/arXiv.2310.04959.

[118] C. Yuan *et al.*, *A Survey of Graph Transformers: Architectures, Theories and Applications*, Feb. 2025. DOI: 10.48550/arXiv.2502.16533.

[119] H. Yuan, H. Yu, S. Gui, and S. Ji, "Explainability in Graph Neural Networks: A Taxonomic Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–19, 2022. DOI: 10.1109/TPAMI.2022.3204236.

[120] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, *GraphSAINT: Graph Sampling Based Inductive Learning Method*, Jul. 2019. DOI: 10.48550/arXiv.1907.04931.

[121] D. Zhang *et al.*, *Mixture of Experts in Large Language Models*, Jul. 2025. DOI: 10.48550/arXiv.2507.11181.

[122] Q. Zhang *et al.*, *A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models*, Jan. 2025. DOI: 10.48550/arXiv.2501.13958.

[123] Z. Zhang, A. Zhang, M. Li, and A. Smola, *Automatic Chain of Thought Prompting in Large Language Models*, Oct. 2022. DOI: 10.48550/arXiv.2210.03493.

[124] B. Zhao, L. G. Foo, P. Hu, C. Theobalt, H. Rahmani, and J. Liu, *LLM-based Agentic Reasoning Frameworks: A Survey from Methods to Scenarios*, Aug. 2025. DOI: 10.48550/arXiv.2508.17692.

[125] H. Zhao *et al.*, *Explainability for Large Language Models: A Survey*, Nov. 2023. DOI: 10.48550/arXiv.2309.01029.

[126] W. X. Zhao *et al.*, *A Survey of Large Language Models*, Mar. 2023. DOI: 10.48550/arXiv.2303.18223.

[127] L. Zheng *et al.*, *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, Dec. 2023. DOI: 10.48550/arXiv.2306.05685.

[128] Y. Zheng, L. Yi, and Z. Wei, *A survey of dynamic graph neural networks*, Apr. 2024. DOI: 10.48550/arXiv.2404.18211.

[129] J. Zhou *et al.*, *Graph Neural Networks: A Review of Methods and Applications*, Dec. 2018. DOI: 10.48550/arXiv.1812.08434.